МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования «Крымский федеральный университет
имени В.И. Вернадского»
Физико-технический институт (структурное подразделение)
Кафедра компьютерной инженерии и моделирования

ПРАКТИКУМ
ПО УЧЕБНОЙ ДИСЦИПЛИНЕ
«СТРУКТУРЫ И АЛГОРИТМЫ ОБРАБОТКИ ДАННЫХ»

для направлений подготовки
09.03.01 "Информатика и вычислительная техника",
09.03.04 «Программная инженерия»

квалификация бакалавр

Симферополь 2020

Практикум по учебной дисциплине «Структуры и алгоритмы обработки данных». – Симферополь: ФГАОУ ВО «Крымский федеральный университет имени В. И. Вернадского», 2020. – 90 стр.

Разработчики - Горская И.Ю., старший преподаватель кафедры компьютерной инженерии и моделирования

Утверждено на заседании кафедры компьютерной инженерии и моделирования, протокол от 15.01.2020г. № 5

Заведующий кафедрой компьютерной инженерии и моделирования  (Милюков В.В.)

Издается по решению Методического совета Физико-технического института ФГАОУ ВО «КФУ им. В.И. Вернадского»
(протокол №5 от 24.01.2020 г.).

Председатель методической комиссии  (А.Ф. Рыбась)

© Горская И.Ю., 2020
© ФГАОУ ВО «Крымский федеральный университет имени В.И. Вернадского», 2020

Практические занятия

Тема 1: Системы счисления. Арифметические операции в разных системах счисления. Перевод из одной системы счисления в другую

Теоретические сведения

1. Системы счисления

Привычная для нас десятичная система счисления имеет, по сравнению с другими системами счета, физиологическое преимущество – большинство людей от рождения обладают «встроенным» в собственное тело калькулятором – двумя руками и двумя ногами, и, соответственно, двадцатью пальцами. В «живом» калькуляторе пальцы могут служить одновременно числами (1-20), разрядами (100 - 1020) и индикаторами (включено/выключено, т.е. согнут/разогнут). Наглядность представления чисел предопределило успех этой системы счисления в древности, поскольку позволяло осуществлять, скажем, торговлю между группами людей, говоривших на разных языках. Высокая разрядность «биологического» калькулятора (до 1020) не использовалась в полной мере, и двадцатеричные системы счисления в истории человечества встречались редко (чукчи, майя, баски, кельтские народы в прежние времена, народы Франции - достаточно вспомнить отголоски ее во французских числительных - роман Виктора Гюго "93 год" по-французски "Quatre-vingts-treize" - "четыре двадцатки тринадцать"). Зато существовали другие, более простые системы счета – например, пятеричная у китайцев (по количеству пальцев на одной руке или ноге) или двенадцатеричная у англичан.

За всю свою историю человечество смогло создать только два типа систем счисления или записи цифр – ***позиционную*** и ***непозиционную***, или знаковую. Непозиционные системы записи цифр появились раньше позиционных.

Непозиционная система счисления применялась в Древнем Риме – для записи цифр использовались буквы латинского алфавита, обозначавшие определенные количества:

I - единица
V - пять
X - десять
L - пятьдесят
C - сто
D - пятьсот
M - тысяча

В этой системе цифры записывались так, чтобы при суммировании цифровых эквивалентов букв получалось требуемое число, например:

MDCLXVI = 1000+500+100+50+10+5+1=1666

Если младшая цифра стоит перед старшей, то эта цифра вычитается:

MCDXLIV = 1000-100+500-10+50-1+5=1444

Возникает ощущение, что одну и ту же цифру можно записать разными способами – например, **XIX** и **XVIIII**, однако специальное правило запрещает запись буквы **I** в цифре более трех раз подряд.

Позиционная система счисления подразумевает более сложный уровень абстракции – для записи цифр используется базовый набор символов, число которых составляет ***основание*** системы счисления. Место каждого символа в числе называется ***позицией***, а номер позиции символа (за вычетом единицы) называется ***разрядом***. Разряды увеличиваются, начиная с нулевого: нулевой, первый, второй и т.д., причем нулевой называется ***младшим*** разрядом, а последний – ***старшим*** разрядом.

В ***позиционных системах*** счисления любое положительное число может быть записано при помощи формулы, составить которую можно, введя следующие обозначения:

Пусть p - основание системы счисления, $\mathbf{A}_p$ - количественный эквивалент числа **A**, состоящего из n цифр $\mathbf{a}_k$, где k =0, …, n-1 . Тогда число **A** можно представить как последовательность цифр $\mathbf{A}_p = \mathbf{a}_{n-1}\mathbf{a}_{n-2}...\mathbf{a}_1\mathbf{a}_0$, причем всегда $\mathbf{a}_k < p$.

В общем случае количественный эквивалент любого положительного числа в позиционной системе счисления можно представить в виде:

$$\mathbf{A}_p = \mathbf{a}_{n-1}\cdot p^{n-1}+\mathbf{a}_{n-2}\cdot p^{n-2} + ... + \mathbf{a}_1\cdot p^1+\mathbf{a}_0\cdot p^0, \qquad (1.1)$$

где **a** -цифра данной системы счисления , n - номер старшего разряда числа.

Проанализировав это выражение, можно сформулировать следующее общее правило: ***количественный эквивалент числа в некоторой позиционной системе счисления равен сумме произведений количественных значений цифр и степеней основания, показатели которых равны номерам разрядов, причем нумерация разрядов начинается с ноля.***

В привычной для нас десятичной системе счисления любое положительное число может быть представлено по формуле (1.1) аналогично следующему примеру:

$$\mathbf{1254698357 = 1\cdot10^9+2\cdot10^8+5\cdot10^7+4\cdot10^6+6\cdot10^5+9\cdot10^4+8\cdot10^3+3\cdot10^2+5\cdot10^1+7\cdot10^0}$$

1.1. Двоичная система счисления

Как известно, персональные компьютеры состоят из полупроводниковых интегральных микросхем различного уровня сложности, совместная работа которых обеспечивается, с одной стороны, согласованностью электрических и частотных характеристик (для работы устройств на «физическом» уровне), а с другой стороны, строгим соблюдением определенных логических правил (для выполнения устройствами задач «логического» уровня). Современные технологии изготовления цифровых микросхем базируются на двух типах логических вентилей, или переключателей (подробнее об этом будет изложено в курсах «Цифровые ЭВМ», «Компьютерная микроэлектроника», и др.), а именно,

TTL (**ТТЛ** *Transistor-Transistor Logic* – транзисторно-транзисторная логика) и **CMOS** (*Complementary Metal-Oxide Semiconductor, (перевод - комплементарный металло-оксид-полупроводник),* **КМОП** – комплементарная логика на транзисторах с изолированным затвором) – структурах. Микросхемы **TTL** по быстродействию несколько выше, а **CMOS** имеют больший размах сигнала, малые входные токи и малое потребление питания.

На «физическом» уровне напряжение питания логических микросхем – постоянное напряжение +5 В относительно общего провода – шины **GND** (от «**Gr**ou**ND**» - земля). В **TTL**-логике различают входы, выходы (обычные, тристабильные и с открытым коллектором) и двунаправленные выводы. Переключения и изменения состояния логических элементов зависит от уровня логических сигналов на входе, каковыми являются напряжения 0,36-0,5 В (***"логический ноль"***) и 2,4…2,7 В (***"логическая единица"***). Порогом переключения состояния является уровень сигнала 1,3-1,4 В. Напряжение ниже этого порога воспринимается как ***низкий*** уровень, выше – как ***высокий***. Состояние свободного (неподключенного) входа **TTL**-микросхемы рассматривается как ***высокоуровневое***.

Из всего вышеизложенного можно сделать вывод, что ***«физический»*** уровень работы интегральных микросхем, составляющих персональный компьютер, предоставляет ***«логическому»*** уровню базис из двух состояний: «есть сигнал» - «нет сигнала». Логично принять присутствие сигнала за логическую единицу, а его отсутствие – за логический ноль. Именно этот факт и привел к широкому использованию ***двоичной системы*** счисления в вычислительной технике, в которой имеются только две цифры – {0 , 1}, т.е. основание $\boldsymbol{p}_{(2)} = 2$. Иногда эти двоичные числа называют ***битами*** (от англ. ***binary digit***). По умолчанию считается, что «**0**» - «выключено» (**LOW** signal), а «**1**» - «включено» (**HIGH** signal).

Согласно формуле (1.1), количественный эквивалент некоторого целого положительного ***n*** - значного числа в двоичной системе отсчета равен:

$$\mathbf{A}_{2} = \mathbf{a}_{n-1}\cdot 2^{n-1} + \mathbf{a}_{n-2}\cdot 2^{n-2} + \ldots + \mathbf{a}_{1}\cdot 2^{1} + \mathbf{a}_{0}\cdot 2^{0}, \qquad (1.2)$$

Например,

$$\mathbf{1011001 = (1\cdot 2^6)+(0\cdot 2^5)+(1\cdot 2^4)+(1\cdot 2^3)+(0\cdot 2^2)+(0\cdot 2^1)+(1\cdot 2^0) =64+16+8+1 = 89_{10}}$$

1.2. Восьмеричная система счисления

Одними из самых первых персональных компьютеров таких фирм, как Apple II, Commodore 64, TRS-80, Motorola и IBM использовали 8-**битные** микропроцессоры, которые могли обрабатывать по восемь **битов** информации за один такт. Для обработки более 8 **битов** они выполняли дополнительные операции.

Битом называют отдельную цифру в двоичной системе исчисления, тетрадой – группу из 4 **бит**. Группа из 8 **битов**, называемая **байт**, вошла в «плоть и кровь» логической архитектуры процессоров всех последующих поколений микропроцессоров (в том числе 16, 32 и 64 разрядных). **Байт** может представлять диапазон десятичных значений от 0_{10} (00000000_2) до 255_{10} (11111111_2). Де факто **байт** стали представлять основной единицей обработки количества информации.

В состав любого микропроцессора входит аккумулятор, разрядность которого обычно совпадает с разрядностью микропроцессора (процессора). Вначале аккумуляторы были 8-ми разрядными, в них один байт составлял одно слово (длина слова в этом случае – 8 бит). Вообще, слово – одна группа обрабатываемых бит, единое выражение или одна команда микропроцессора (процессора). Восьмиразрядный процессор переносит и помещает все данные группами из 8 бит, которые передаются восемью проводниками, составляющими шину данных, 16-ти разрядный – группами по 16 бит (у него длина слова 2 байта), и т.д.

Восьмеричная система исчисления обладает базисом из восьми цифр {0 , 1 , 2, 3 , 4, 5, 6, 7}, так как ее основание $\boldsymbol{p} = 8$. Для отличия от десятичной системы после цифр часто ставят латинскую букву Q – 327Q Согласно формуле (1.1), количественный эквивалент целого положительного числа в восьмеричной системе отсчета равен:

$$\mathbf{A}_{\boldsymbol{8}} = \mathbf{a}_{\boldsymbol{n-1}}\cdot\boldsymbol{8}^{n-1}+\mathbf{a}_{\boldsymbol{n-2}}\cdot\boldsymbol{8}^{n-2} + \ldots + \mathbf{a}_{\boldsymbol{1}}\cdot\boldsymbol{8}^{1}+\mathbf{a}_{\boldsymbol{0}}\cdot\boldsymbol{8}^{0} \qquad (1.3)$$

Например,

$$\mathbf{12345670Q = (1\cdot 8^7)+(2\cdot 8^6)+(3\cdot 8^5)+(4\cdot 8^4)+(5\cdot 8^3)+(6\cdot 8^2)+(7\cdot 8^1)+(0\cdot 8^0) =}$$
$$\mathbf{2097152+524288+98304+16384+2560+384+56 = 2739128_{10}}$$

1.3. Десятичная система счисления

Десятичная система счисления - наиболее привычная для нас, поскольку используется нами при повседневном счете. По всей видимости, исторически распространенность этой системы исчисления обусловлено наличием 10 пальцев на обеих руках, активно используемых в качестве "биологического калькулятора". Это подтверждается самими названиями цифр в разных языках: в латинском ***digiti*** (пальцевые числа 1,2,...9) от ***digitus*** (палец), ***articuli*** (суставные числа, т.е. полные десятки), ***compositi*** (составные, из десятков и единиц), в английском - ***digits***, в итальянском - ***unitates*** (единицы), ***deceni*** (десятки), в греческом - ***monadici*** (единицы), ***decades*** (десятки), в прежнем русском - ***персты*** (единицы), ***составы*** (десятки), ***сочинения*** (числа из десятков и единиц), ноль - ***незачто***.

Хорошо известно, что данная система исчисления использует следующий базовый набор из 10 цифр {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}, поскольку ее основание $\boldsymbol{p} = 10$.

Согласно формуле (1.1), количественный эквивалент целого положительного числа в десятичной системе отсчета равен:

$$\mathbf{A}_{10} = \mathbf{a}_{n-1}\cdot 10^{n-1}+\mathbf{a}_{n-2}\cdot 10^{n-2} + \ldots + \mathbf{a}_1\cdot 10^1+\mathbf{a}_0\cdot 10^0 \qquad (1.4)$$

Например,

$$\mathbf{1234567890 =}$$
$$\mathbf{(1\cdot 10^9)+(2\cdot 10^8)+(3\cdot 10^7)+(4\cdot 10^6)+(5\cdot 10^5)+(6\cdot 10^4)+(7\cdot 10^3)+(8\cdot 10^2)+(9\cdot 10^1)+(0\cdot 10^0)}$$
$$\mathbf{= 1234567890_{10}}$$

1.4. Шестнадцатеричная система счисления

Шестнадцатеричная система исчисления менее привычна для нас, поскольку не используется нами при повседневном счете (конечно, если вы

не программист). Шестнадцатеричная система исчисления, так же как и восьмеричная, широко используется в компьютерной науке из-за легкости перевода в нее двоичных чисел. При шестнадцатеричной записи числа получаются более компактными.

В шестнадцатеричной системе счисления в качестве базового набора используются цифры от 0 до 9 и шесть первых латинских букв – A (10), B (11), C (12), D (13), E (14), F (15), поскольку ее основание $\boldsymbol{p} = 16$. Для отличия от остальных систем исчисления после цифр часто ставят латинскую букву H (иногда h) – 3FBC27H или 3FBC27h.

Согласно формуле (1.1), количественный эквивалент целого положительного числа в шестнадцатеричной системе отсчета равен:

$$\mathbf{A}_{16} = \mathbf{a}_{n-1}\cdot 16^{n-1} + \mathbf{a}_{n-2}\cdot 16^{n-2} + \ldots + \mathbf{a}_{1}\cdot 16^{1} + \mathbf{a}_{0}\cdot 16^{0}, \qquad (1.5)$$

Например,

$$\mathbf{ABCDEF12h} =$$
$$= (10\cdot 16^7)+(11\cdot 16^6)+(12\cdot 16^5)+(13\cdot 16^4)+(14\cdot 16^3)+(15\cdot 16^2)+(1\cdot 16^1)+(2\cdot 16^0) =$$
$$= 2882400018_{10}$$

Максимальное двухразрядное число, которое можно получить с помощью шестнадцатеричной записи - это **FF**.

$$\mathbf{FF = 15 * 16^1 + 15 * 16^0 = 240 + 15 = 255}.$$

255 – это максимальное значение одного байта, равного 8 битам: **1111 1111 = FF**. Поэтому с помощью шестнадцатеричной системы счисления очень удобно кратко (с помощью двух цифр-знаков) записывать значения байтов. Внимание! Состояний у 8-ми битного байта может быть 256, однако максимальное значение – 255. Не забывайте про 0 – это как раз 256-е состояние.

1.5. Двоично-десятичная система счисления

Поскольку человеку наиболее привычны представление и арифметика в десятичной системе счисления, а для компьютера - двоичное представление

и двоичная арифметика, была введена компромиссная система ***двоично-десятичной записи*** чисел. Такая система чаще всего применяется там, где существует необходимость частого использования процедуры десятичного ввода-вывода. (электронные часы, калькуляторы, АОНы, и т.д.). В таких устройствах не всегда целесообразно предусматривать универсальный микрокод перевода двоичных чисел в десятичные и обратно по причине небольшого объема программной памяти.

Принцип построения этой системы достаточно прост: каждая десятичная цифра преобразуется прямо в свой десятичный эквивалент из 4 бит, например: **3691_{10}=0011 0110 1001 0001_{DEC}:**

Десятичное число	**3**	**6**	**9**	**1**
Двоично-десятичное число	0011	0110	1001	0001

Преобразуем двоично-десятичное число 1000 0000 0111 0010 в его десятичный эквивалент.

Каждая группа из 4 бит преобразуется в её десятичный эквивалент.
Получим **1000 0000 0111 0010_{DEC} = 8072_{10}:**

Двоично-десятичное число	**1000**	**0000**	**0111**	**0010**
Десятичное число	8	0	7	2

Микропроцессоры используют чистые двоичные числа, однако понимают и команды преобразования в двоично-десятичную запись. Полученные двоично-десятичные числа легко представимы в десятичной записи, более понятной людям.

2. Перевод чисел из одной системы счисления в другую

2.1. Перевод чисел из двоичной системы в десятичную

Задача перевода чисел из двоичной системы счисления в десятичную чаще всего возникает уже при обратном преобразовании вычисленных либо обработанных компьютером значений в более понятные пользователю десятичные цифры. Алгоритм перевода двоичных чисел в десятичные достаточно прост (его иногда называют *алгоритмом замещения*):

Для перевода двоичного числа в десятичное необходимо это число представить в виде суммы произведений степеней основания двоичной системы счисления на соответствующие цифры в разрядах двоичного числа.

Например, требуется перевести двоичное число **10110110** в десятичное. В этом числе **8** цифр и **8** разрядов (разряды считаются, начиная с нулевого, которому соответствует младший бит). В соответствии с уже известным нам правилом представим его в виде суммы степеней с основанием 2:

$$\mathbf{10110110_2 = (1\cdot 2^7)+(0\cdot 2^6)+(1\cdot 2^5)+(1\cdot 2^4)+(0\cdot 2^3)+(1\cdot 2^2)+(1\cdot 2^1)+(0\cdot 2^0) =}$$

$$\mathbf{= 128+32+16+4+2 = 182_{10}}$$

Из этого примера видно, в частности, что десятичная система счисления более компактно отображает числа - 3 цифры (т.е. бита) вместо 8 цифр в двоичной системе счисления. Для вычислений "вручную" и решения примеров и контрольных заданий вам могут пригодиться таблицы степеней оснований изучаемых систем счисления (2, 8, 10, 16), приведенные в Приложении.

2.2. Перевод чисел из восьмеричной системы в десятичную

Алгоритм перевода чисел из восьмеричной в десятичную систему счисления аналогичен предыдущему алгоритму перевода чисел из двоичной системы в десятичную. Различие состоит лишь в том, что для восьмеричной системы счисления основанием является число **8**, а правило перевода в данном случае может быть сформулировано в следующем виде:

Для перевода восьмеричного числа в десятичное необходимо это число представить в виде суммы произведений степеней основания восьмеричной системы счисления на соответствующие цифры в разрядах восьмеричного числа.

Например, требуется перевести восьмеричное число **2357** в десятичное. В этом числе **4** цифры и **4** разряда (разряды считаются, начиная с нулевого, которому соответствует младший бит). В соответствии с уже известным нам правилом представим его в виде суммы степеней с основанием **8**:

$$2357_8 = (2 \cdot 8^3)+(3 \cdot 8^2)+(5 \cdot 8^1)+(7 \cdot 8^0) = 2 \cdot 512 + 3 \cdot 64 + 5 \cdot 8 + 7 \cdot 1 = 1263_{10}$$

Для вычислений "вручную" и решения примеров и контрольных заданий вам могут пригодиться таблицы степеней оснований изучаемых систем счисления (2, 8, 10, 16), приведенные в Приложении.

2.3. Перевод чисел из шестнадцатеричной системы в десятичную

После изучения предыдущего раздела переформулировать алгоритм перевода чисел из шестнадцатеричной в десятичную систему счисления не составляет никакого труда. Помнить следует лишь о том, что для шестнадцатеричной системы счисления основанием является число **16**, и правило перевода в данном случае может быть сформулировано в следующем виде:

Для перевода шестнадцатеричного числа в десятичное необходимо это число представить в виде суммы произведений степеней основания шестнадцатеричной системы счисления на соответствующие цифры в разрядах шестнадцатеричного числа.

Например, требуется перевести шестнадцатеричное число **F45ED23C** в десятичное. В этом числе **8** цифр и **8** разрядов (помним, что разряды считаются, начиная с нулевого, которому соответствует младший бит). В соответствии с вышеуказанным правилом представим его в виде суммы степеней с основанием **16**:

$$\mathbf{F45ED23C_{16} =}$$
$$\mathbf{= (15\cdot16^7)+(4\cdot16^6)+(5\cdot16^5)+(14\cdot16^4)+(13\cdot16^3)+(2\cdot16^2)+(3\cdot16^1)+(12\cdot16^0) =}$$
$$\mathbf{= 4099854908_{10}}$$

Для вычислений "вручную" и решения примеров и контрольных заданий вам могут пригодиться таблицы степеней оснований изучаемых систем счисления (2, 8, 10, 16), приведенные в Приложении.

2.4. Перевод чисел из десятичной системы в двоичную

Для перевода чисел из десятичной системы счисления в двоичную используют так называемый "алгоритм замещения", состоящий из следующей последовательности действий:

1. Делим десятичное число **A** на **2**. Частное **Q** запоминаем для следующего шага, а остаток **a** записываем как ***младший*** бит двоичного числа.

2. Если частное **Q** не равно **0**, принимаем его за новое делимое и повторяем процедуру, описанную в шаге 1. Каждый новый остаток (**0** или **1**) записывается в разряды двоичного числа в направлении от ***младшего*** бита к ***старшему***.

3. Алгоритм продолжается до тех пор, пока в результате выполнения шагов 1 и 2 не получится частное **Q** = **0** и остаток **a** = **1**.

Например, требуется перевести десятичное число **247** в двоичное. В соответствии с приведенным алгоритмом получим:

$\mathbf{247_{10} : 2 = 123_{10}}$
$\mathbf{247_{10} - 246_{10} = 1,}$ остаток **1** записываем в **МБ** двоичного числа
$\mathbf{123_{10} : 2 = 61_{10}}$
$\mathbf{123_{10} - 122_{10} = 1,}$ остаток **1** записываем в следующий после **МБ** разряд двоичного числа
$\mathbf{61_{10} : 2 = 30_{10}}$
$\mathbf{61_{10} - 60_{10} = 1,}$ остаток **1** записываем в старший разряд двоичного числа

30_{10} : 2 = 15_{10}
30_{10} - 30_{10} = 0, остаток **0** записываем в старший разряд двоичного числа
15_{10} : 2 = 7_{10}
15_{10} - 14_{10} = 1, остаток **1** записываем в старший разряд двоичного числа
7_{10} : 2 = 3_{10}
7_{10} - 6_{10} = 1, остаток **1** записываем в старший разряд двоичного числа
3_{10} : 2 = 1_{10}
3_{10} - 2_{10} = 1, остаток **1** записываем в старший разряд двоичного числа
1_{10} : 2 = 0_{10}, остаток **1** записываем в старший разряд двоичного числа

Таким образом, искомое двоичное число равно **11110111_2**.

2.5. Перевод чисел из десятичной системы в восьмеричную

Для перевода чисел из десятичной системы счисления в восьмеричную используют тот же "алгоритм замещения", что и при переводе из десятичной системы счисления в двоичную, только в качестве делителя используют **8**, основание восьмеричной системы счисления:

1. Делим десятичное число **A** на **8**. Частное **Q** запоминаем для следующего шага, а остаток **a** записываем как ***младший*** бит восьмеричного числа.

2. Если частное **Q** не равно **0**, принимаем его за новое делимое и повторяем процедуру, описанную в шаге 1. Каждый новый остаток записывается в разряды восьмеричного числа в направлении от ***младшего*** бита к ***старшему***.

Алгоритм продолжается до тех пор, пока в результате выполнения шагов 1 и 2 не получится частное **Q** = **0** и остаток **a** меньше **8**.

Например, требуется перевести десятичное число **3336** в восьмеричное. В соответствии с приведенным алгоритмом получим:

3336_{10} : 8 = 417_{10}
3336_{10} - 3336_{10} = 0, остаток 0 записываем в МБ восьмеричного числа
417_{10} : 8 = 52_{10}
417_{10} - 416_{10} = 1, остаток 1 записываем в следующий после МБ разряд восьмеричного числа
52_{10} : 8 = 6_{10}
52_{10} - 48_{10} = 4, остаток 4 записываем в старший разряд восьмеричного числа
6_{10} : 8 = 0_{10}, остаток 0, записываем 6 в самый старший разряд восьмеричного числа

Таким образом, искомое восьмеричное число равно **6410_8.**

2.6. Перевод чисел из десятичной системы в шестнадцатеричную

Для перевода чисел из десятичной системы счисления в шестнадцатеричную используют тот же "алгоритм замещения", что и при переводе из десятичной системы счисления в двоичную и восьмеричную, только в качестве делителя используют **16**, основание шестнадцатеричной системы счисления:

1. Делим десятичное число **A** на **16**. Частное **Q** запоминаем для следующего шага, а остаток **a** записываем как ***младший*** бит шестнадцатеричного числа.

2. Если частное **Q** не равно **0**, принимаем его за новое делимое и повторяем процедуру, описанную в шаге 1. Каждый новый остаток записывается в разряды шестнадцатеричного числа в направлении от ***младшего*** бита к ***старшему***.

3. Алгоритм продолжается до тех пор, пока в результате выполнения шагов 1 и 2 не получится частное **Q** = **0** и остаток **a** меньше **16**.

Например, требуется перевести десятичное число **32767** в шестнадцатеричное. В соответствии с приведенным алгоритмом получим:

32767_{10} : 16 = 2047_{10}
32767_{10} - 32752_{10} = 15, остаток 15 в виде F записываем в МБ шестнадцатеричного числа
2047_{10} : 16 = 127_{10}
2047_{10} - 2032_{10} = 15, остаток 15 в виде F записываем в следующий после МБ разряд шестнадцатеричного числа
127_{10} : 16 = 7_{10}
127_{10} - 112_{10} = 15, остаток 15 в виде F записываем в старший разряд шестнадцатеричного числа
7_{10} : 16 = 0_{10}, остаток 7 записываем в старший разряд шестнадцатеричного числа

Таким образом, искомое шестнадцатеричное число равно **$7FFF_{16}$.**

2.7. Перевод чисел из десятичной системы в двоично-десятичную

Разработчики программного и аппаратного обеспечения вычислительной техники сталкиваются не только с проблемами представления чисел и операций с ними, но и с проблемами интерпретации и вывода результатов машинной обработки данных (в частности, вычислений) в удобной для пользователя или оператора ЭВМ или микроЭВМ форме.

Разрабатывая логику организации вычислений с помощью ЭВМ, поневоле приходится решать три задачи: ВВОД ДАННЫХ, ОБРАБОТКУ ДАННЫХ и ВЫВОД ДАННЫХ. В простейшем случае ВВОД ДАННЫХ осуществляется пользователем или оператором ЭВМ, а ОБРАБОТКА ДАННЫХ и ВЫВОД ДАННЫХ - самой ЭВМ. На всех этих этапах происходит преобразование данных - пользователю удобно вводить числа в десятичной системе счисления, а ЭВМ вынуждена преобразовывать их в двоичные числа, производить вычисления на стадии ОБРАБОТКИ ДАННЫХ

и снова преобразовывать полученные результаты в десятичную систему счисления, организуя ВЫВОД ДАННЫХ в понятном для пользователя виде.

Мы подробно рассматривали перевод чисел из десятичной системы счисления в двоично-десятичную и обратно в разделе «Двоично-десятичная система счисления».

2.8. Перевод чисел из двоичной системы в восьмеричную и шестнадцатеричную

При обработке данных и вычислениях одной из наиболее часто встречающихся задач является перевод чисел из одной системы счисления в другую. Рассмотрим простейшие алгоритмы перевода положительных чисел из двоичной системы в восьмеричную и шестнадцатеричную.

Пример 1. Пусть требуется перевести двоичное число **$1010110110011011011110010101100101 1_2$** в восьмеричную систему счисления. Для этого следует разбить это двоичное число на *триады*, начиная с ***младшего бита*** (**МБ**).

Двоичная система	Восьмеричная система	Двоичная система	Восьмеричная система
000	**0**	**100**	**4**
001	**1**	**101**	**5**
010	**2**	**110**	**6**
011	**3**	**111**	**7**

Получим:

$010\ 101\ 101\ 100\ 110\ 110\ 111\ 100\ 101\ 011\ 001\ 011_2$

Если *старшая триада* не заполнена до конца, следует дописать в ее старшие разряды нули, как в нашем случае. После этого необходимо

заменить двоичные триады, начиная с младшей, на числа, равные им в восьмеричной системе:

$$\mathbf{2\ 5\ 5\ 4\ 6\ 6\ 7\ 4\ 5\ 3\ 1\ 3_8}$$

Таким образом,

$$\mathbf{10101101100110110111100101011001011_2=255466745313_8}$$

Аналогично поступаем при переводе чисел из двоичной системы счисления в шестнадцатеричную, но разбиение двоичного числа производим на *тетрады*. Для ***примера 2*** будем использовать то же двоичное число, что и при переводе в восьмеричную систему счисления. Число разбивается на группы по четыре разряда, начиная с конца. В случае, если количество разрядов не делится нацело, то первая четверка дописывается нулями впереди. Каждой четверке соответствует цифра шестнадцатеричной системы счисления:

Двоичное число	Шестнадцатеричное число	Двоичное число	Шестнадцатеричное число
0000	0	1000	8
0001	1	1001	9
0010	2	1010	A
0011	3	1011	B
0100	4	1100	C
0101	5	1101	D
0110	6	1110	E
0111	7	1111	F

$$\mathbf{0101\ 0110\ 1100\ 1101\ 1011\ 1100\ 1010\ 1100\ 1011_2}$$

Заменяя двоичные тетрады на их шестнадцатеричные значения, получим искомое шестнадцатеричное число:

$$\mathbf{10101101100110110111100101011001011_2=56CDBCACB_{16}}$$

Пример 3. Перевести двоичное число **10001100101** в шестнадцатеричное. Получим:

$$\mathbf{10001100101_2 = 0100\ 1100\ 0101_2 = 4\ C\ 5_{16} = 4C5_{16}}$$

Если потребуется, то число $\mathbf{4C5_{16}}$ можно перевести в десятичную систему счисления следующим образом (С следует заменить на соответствующее данному символу число в десятичной системе счисления – это 12):

$$\mathbf{4C5_{16} = 4 * 16^2 + 12 * 16^1 + 5 * 16^0 = 4 * 256 + 192 + 5 = 1221_{10}}$$

Очевидно, что разбиения на триады и тетрады связаны со степенями двойки (для триады, при переводе в восьмеричную систему, $\mathbf{2^3}$, а для тетрады, при переводе в шестнадцатеричную, $\mathbf{2^4}$)). Сравнительные таблицы соответствия чисел в различных системах счисления можно найти в Приложении.

2.9. Перевод чисел из восьмеричной системы в двоичную и шестнадцатеричную

Алгоритм перевода чисел из одной системы счисления в другую наиболее прост в том случае, когда одно из оснований этих систем является степенью другой, как, например, в случае двоичной и восьмеричной систем счисления. В таком случае алгоритм перевода состоит в простой замене чисел одной системы на равные им числа другой системы счисления (в случае положительных чисел). На начальном этапе удобно и полезно воспользоваться таблицей соответствия, приведенной в Приложении.

Пусть требуется перевести восьмеричное число $\mathbf{2473_8}$ в двоичное число. Воспользовавшись Таблицей соответствия из Приложения, получим:

$$\mathbf{2473_8 = 10100111011_2},$$

поскольку $\mathbf{2_8 = 010_2}$, $\mathbf{4_8 = 100_2}$, $\mathbf{7_8 = 111_2}$... Следует помнить, что восьмеричное число кодируется тремя битами, и выписывать триады нужно полностью.

Исключением из этого правила может служить только старшая триада, в которой старший бит (**СБ**) равен нулю.

Сложнее обстоит дело при переводе чисел из восьмеричной системы в шестнадцатеричную. Обычно, вначале переводят восьмеричное число в двоичное, а затем уже в шестнадцатеричное по алгоритму, описанному в соответствующем разделе. Для рассмотренного выше примера имеем:

$$\mathbf{2473_8 = 10100111011_2 = 0101\ 0011\ 1011_2 = 53B_{16}}$$

2.10. Перевод чисел из шестнадцатеричной системы в двоичную и восьмеричную

Алгоритм перевода чисел из шестнадцатеричной системы счисления в двоичную крайне прост. Необходимо только заменить каждую цифру шестнадцатеричного числа ее эквивалентом в двоичной системе счисления (в случае положительных чисел). Как и в предыдущих параграфах, удобно и полезно воспользоваться таблицей соответствия, приведенной в Приложении. Отметим только, что каждое шестнадцатеричное число следует заменять двоичным, дополняя его до 4 разрядов (в сторону старших разрядов).

Пусть требуется перевести шестнадцатеричное число $\mathbf{F1_{16}}$ в двоичное число. Воспользовавшись Таблицей соответствия, получим:

$$\mathbf{F1_{16} = 11110001_2},$$

поскольку $\mathbf{F_{16} = 1111_2}$, $\mathbf{1_{16} = 0001_2}$. Этот пример иллюстрирует тот факт, что следует дополнять младшие разряды до 4 разряда в двоичном числе. Естественно, дополнять старший разряд двоичного числа до 4 старших битов нулями не имеет смысла, другими словами пишут

$$\mathbf{1F_{16} = 11111_2}, \text{ а не } \mathbf{00011111_2}$$

Обычно, при переводе чисел из шестнадцатеричной в восьмеричную систему счисления вначале шестнадцатеричное число переводят в двоичное,

затем разбивают его на триады, начиная с младшего бита, а потом заменяют триады соответствующими им эквивалентами в восьмеричной системе (аналогично алгоритму, описанному в разделе «Перевод чисел из двоичной системы в восьмеричную и шестнадцатеричную». Для рассмотренных выше примеров имеем:

$$\mathbf{1F_{16} = 11111_2 = 011\ 111_2 = 37_8}$$

$$\mathbf{F1_{16} = 11110001_2 = 011\ 110\ 001_2 = 361_8}$$

Непосредственное преобразование чисел из шестнадцатеричной системы счисления в восьмеричную требует выполнения арифметических действий в этой системе счисления. Об этом речь пойдет позже. Следует отметить, что программная реализация вышеприведенного алгоритма проще и надежнее, поскольку при выполнениях операций деления неизбежно возникают дробные числа и переполнения разрядной сетки, необходимость округления, и, как следствие, потеря точности, не говоря уже о скорости выполнения компьютером такого типа алгоритмов.

2.11. Дробные числа в двоичной системе счисления

В любой системе счисления нужно уметь представлять не только целые числа, но и дробные. С математической точки зрения это ординарная задача, которая давно решена. Однако с точки зрения компьютерной техники это далеко не тривиальная проблема, во многом связанная с архитектурой компьютера. Ресурсы компьютеров не бесконечны, и основной трудностью является представление периодических и непериодических дробей.

Следовательно, такие дроби следует округлять, задавать класс точности участвующих (и могущих появиться в результате вычислений!) чисел без потери точности вычислений, а также следить за тем, чтобы потеря точности не произошла при переводе чисел из одной системы счисления в другую. Особенно важно аккуратно производить вычисления при операциях с плавающей точкой.

Запишем формулу представления дробного числа в позиционной системе счисления:

$$\mathbf{A}_p = \mathbf{a}_{n-1}\cdot p^{n-1}+\mathbf{a}_{n-2}\cdot p^{n-2} + \ldots + \mathbf{a}_1\cdot p^1+\mathbf{a}_0\cdot p^0 +\mathbf{a}_{-1}\cdot p^{-1}+\mathbf{a}_{-2}\cdot p^{-2} + \ldots + \mathbf{a}_{-m}\cdot p^{-m}, \quad (2.1)$$

В случае десятичной системы счисления получим:

$$\mathbf{24{,}73_2} = \mathbf{2\cdot 10^1+4\cdot 10^0+7\cdot 10^{-1}+3\cdot 10^{-2}}$$

Перевод дробного числа из двоичной системы счисления в десятичную производится по следующей схеме:

$$\mathbf{101101{,}101_2} = \mathbf{1\cdot 2^5+0\cdot 2^4+1\cdot 2^3+1\cdot 2^2+0\cdot 2^1+1\cdot 2^0+1\cdot 2^{-1}+0\cdot 2^{-2}+1\cdot 2^{-3}=45{,}625}$$

Перевод дробного числа из десятичной системы счисления в двоичную осуществляется по следующему алгоритму:

- Вначале переводится целая часть десятичной дроби в двоичную систему счисления;
- Затем дробная часть десятичной дроби умножается на основание двоичной системы счисления;
- В полученном произведении выделяется целая часть, которая принимается в качестве значения первого после запятой разряда числа в двоичной системе счисления;
- Алгоритм завершается, если дробная часть полученного произведения равна нулю или если достигнута требуемая точность вычислений. В противном случае вычисления продолжаются с предыдущего шага.

Пример: Требуется перевести дробное десятичное число 206,116 в дробное двоичное число.

Перевод целой части дает $\mathbf{206_{10}=11001110_2}$ по ранее описанным алгоритмам; дробную часть умножаем на основание **2**, занося целые части произведения в разряды после запятой искомого дробного двоичного числа:

.116 • 2 = **0**.232
.232 • 2 = **0**.464
.464 • 2 = **0**.928
.928 • 2 = **1**.856

.856 • 2 = **1**.712
.712 • 2 = **1**.424
.424 • 2 = **0**.848
.848 • 2 = **1**.696
.696 • 2 = **1**.392
.784 • 2 = **0**.784
и т.д.

Получим: $\mathbf{206{,}116_{10}=11001110{,}0001110110_2}$

2.12. Общие правила перевода чисел из одной системы счисления в другую

Правило 1. Перевод смешанного числа (числа с дробной частью) из p - ичной системы счисления в q - ичную систему счисления, когда имеет место соотношение $p = q^k$ (k − целое положительное число), осуществляется поразрядно. Каждая p - ичная цифра заменяется равным ей k - разрядным числом, записанным в q - ичной системе счисления.

Для упрощения перевода по описанному выше правилу удобно использовать Таблицу соответствия из Приложения.

Пример 1.

а) $A61{,}87C_{(16)} = \underbrace{(1010)}_{A_{(16)}}\underbrace{(0110)}_{6_{(16)}}\underbrace{(0001)}_{1_{(16)}},\underbrace{(1000)}_{8_{(16)}}\underbrace{(0111)}_{7_{(16)}}\underbrace{(1100)}_{C_{(16)}}{}_{(2)}.$

б) $31{,}471_{(8)} = \underbrace{(011)}_{3_{(8)}}\underbrace{(001)}_{1_{(8)}},\underbrace{(100)}_{4_{(8)}}\underbrace{(111)}_{7_{(8)}}\underbrace{(001)}_{1_{(8)}}{}_{(2)}.$

Обратный перевод из q - ичной системы счисления в p - ичную систему счисления осуществляют, разбивая q - ичную запись числа на группы по k цифр, двигаясь от запятой вправо и влево и заменяя каждую группу цифр ее

p - ичным изображением. При этом если крайние группы окажутся неполными, то их дополняют до k цифр незначащими нулями.

Пример 2.

а) $1110010{,}01101111001_{(2)} = \underbrace{(0111)}_{7_{(16)}}\underbrace{(0010)}_{2_{(16)}},\underbrace{(0110)}_{6_{(16)}}\underbrace{(1111)}_{F_{(16)}}\underbrace{(0010)}_{2_{(16)}}{}_{(2)} = 72{,}6F2_{(16)};$

б) $1001111{,}011100_{(2)} = \underbrace{(001)}_{1_{(8)}}\underbrace{(001)}_{1_{(8)}}\underbrace{(111)}_{7_{(8)}},\underbrace{(011)}_{3_{(8)}}\underbrace{(100)}_{4_{(8)}}{}_{(2)} = 117{,}34_{(8)}.$

Перевод смешанного числа производится отдельно для целой и дробной частей числа.

Правило 2.1. Перевод целой части. Целую часть числа, записанную в p - ичной системе счисления, делят на основание q - ичной системы счисления, записанное в p - ичной системе счисления (все операции производятся по правилам p - ичной системы счисления). Полученное в остатке число является младшей (последней) цифрой в q - ичной записи числа. Полученное частное снова делят на основание q; остаток – предпоследняя цифра в искомой записи числа; и т.д. Операцию деления проводят до тех пор, пока в частном не получат число, меньшее q ; частное – старшая (первая) цифра в q - ичной записи числа.

Пример 3.

а) $345_{(10)} = 159_{(16)}$;

$$\begin{array}{r|l} 345 & 16 \\ \underline{32} & 21 \\ 25 & \underline{16} \\ \underline{16} & \mathbf{5} \\ \mathbf{9} & \end{array} \qquad \begin{array}{l|l} & 16 \\ & \\ & \\ & \mathbf{1} \\ & \end{array}$$

б) $24_{(10)} = 11000_{(2)}$

$$\begin{array}{r|l} 24 & 2 \\ \underline{24} & 12 \\ \mathbf{0} & \underline{12} \\ & \mathbf{0} \end{array} \qquad \begin{array}{l|l} & 2 \\ & 6 \\ & \underline{6} \\ & 3 \\ & \mathbf{0} \end{array} \qquad \begin{array}{l|l} & 2 \\ & 2 \\ & \underline{2} \\ & \mathbf{1} \\ & \mathbf{1} \end{array}$$

Правило 2.2. Перевод дробной части. Дробную часть числа, записанную в p - ичной системе счисления, умножают на основание q - ичной системы счисления, записанное в p - ичной системе счисления. Целая часть произведения будет старшей цифрой изображения дроби (первой после запятой) в q - ичной записи числа. Дробную часть произведения снова умножают на основание q; целая часть – следующая цифра после запятой в q - ичной записи дроби. Процесс продолжают до тех пор, пока дробная часть не станет нулем или будет получено требуемое количество знаков после запятой в дробной записи числа. Целые части записывают в q - ичной системе счисления.

Пример 4.

а) $0{,}2_{(10)} = 0{,}\overline{(0011)}_{(2)}$;

$$
\begin{array}{r}
0{,}2 \\
\underline{\quad 2} \\
\mathbf{0}_{(2)} = \mathbf{0}{,}4 \\
\underline{\quad 2} \\
\mathbf{0}_{(2)} = \mathbf{0}{,}8 \\
\underline{\quad 2} \\
\mathbf{1}_{(2)} = \mathbf{1}{,}6 \\
\underline{\quad 2} \\
\mathbf{1}_{(2)} = \mathbf{1}{,}2 \\
\underline{\quad 2} \\
\mathbf{0}_{(2)} = \mathbf{0}{,}4 \\
\underline{\quad 2} \\
\mathbf{0}_{(2)} = \mathbf{0}{,}8 \\
\ldots\ldots
\end{array}
$$

б) $0{,}12_{(10)} \approx 0{,}1\mathrm{EB}_{(16)}$;

$$
\begin{array}{r}
0{,}12 \\
\underline{\quad 16} \\
\mathbf{1}_{(16)} = \mathbf{1}{,}92 \\
0{,}92 \\
\underline{\quad 16} \\
\mathbf{E}_{(16)} = \mathbf{14}{,}72 \\
0{,}72 \\
\underline{\quad 16} \\
\mathbf{B}_{(16)} = \mathbf{11}{,}52 \\
\ldots\ldots\ldots
\end{array}
$$

в) $0{,}\mathrm{A}_{(16)} = 0{,}625_{(10)}$.

$$
\begin{array}{r}
0{,}\mathrm{A} \\
\underline{\quad \mathrm{A}} \\
\mathbf{6}_{(10)} = \mathbf{6}{,}4 \\
0{,}4 \\
\underline{\quad \mathrm{A}} \\
\mathbf{2}_{(10)} = \mathbf{2}{,}8 \\
0{,}8 \\
\underline{\quad \mathrm{A}} \\
\mathbf{5}_{(10)} = \mathbf{5}{,}0
\end{array}
$$

При переводе смешанного числа результаты для целой и дробной частей, полученные по правилам 2.1, 2.2, записывают, отделяя друг от друга дробной запятой.

Пример 5.

$$345{,}02_{(10)} = 101011001{,}(0011)_{(2)}.$$

$$345_{(10)} = 159_{(16)} = \underbrace{(0001)}_{1_{(16)}}\underbrace{(0101)}_{5_{(16)}}\underbrace{(1001)}_{9_{(16)}}{}_{(2)};$$

$$0{,}2_{(10)} = 0{,}(0011)_{(2)}.$$

$$\begin{array}{r|l} 345 & 16 \\ \underline{32} & 21 \\ 25 & \underline{16} \\ \underline{16} & \mathbf{1} \\ \mathbf{9} & \end{array} \qquad \begin{array}{r|l} 21 & 16 \\ \underline{16} & \mathbf{1} \\ \mathbf{5} & \end{array}$$

$$0{,}2 \;\Big|\; \underline{\quad 2}$$
$$\mathbf{0}_{(2)} = \mathbf{0}{,}4 \;\Big|\; \underline{\quad 2}$$
$$\mathbf{0}_{(2)} = \mathbf{0}{,}8 \;\Big|\; \underline{\quad 2}$$
$$\mathbf{1}_{(2)} = \mathbf{1}{,}6 \;\Big|\; \underline{\quad 2}$$
$$\mathbf{1}_{(2)} = \mathbf{1}{,}2 \;\Big|\; \underline{\quad 2}$$
$$\mathbf{0}_{(2)} = \mathbf{0}{,}4$$
$$\ldots\ldots$$

Правило 3. Перевод чисел в $10-$ичную систему счисления рекомендуется выполнять суммированием с учетом «веса» цифры в числе по формуле:

$$R_{(p)} = a_N p^N + a_{N-1} p^{N-1} + \ldots + a_1 p^1 + a_0 p^0 + a_{-1} p^{-1} + \ldots + a_{-K} p^{-K},$$

или

$$a_N a_{N-1} \ldots a_1 a_0 \,.\, a_{-1} \ldots a_{-K} = a_N p^N + a_{N-1} p^{N-1} + \ldots + a_1 p^1 + a_0 p^0 + a_{-1} p^{-1} + \ldots + a_{-K} p^{-K}$$

Пример 6.

а) $1101{,}111_{(2)} = 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 + 1 \cdot 2^{-1} + 1 \cdot 2^{-2} + 1 \cdot 2^{-3} = 13{,}875_{(10)};$

б) $1A{,}02_{(16)} = 1 \cdot 16^1 + 10 \cdot 16^0 + 0 \cdot 16^{-1} + 2 \cdot 16^{-2} = 26{,}0078125_{(10)}.$

4. Арифметические действия в разных системах счисления

Теперь, когда мы познакомились с разными системами счисления, представлением в них чисел, приступим к изучению собственно арифметики.

Напоминаем, что системы счисления бывают *позиционные* и *непозиционные*. В *позиционных* системах счисления любое число записывается в виде последовательности символов, количественное значение («вес») которых зависит от местоположения в числе, т.е. позиции в записи числа. *Основанием* позиционной системы счисления называется целое число, определяющее количество символов, используемых в ней (обозначим его через p). В позиционной системе счисления с основанием p (p - ичной системе счисления) любое число R может быть представлено в виде:

$$R_{(p)} = a_N p^N + a_{N-1} p^{N-1} + \ldots + a_1 p^1 + a_0 p^0 + a_{-1} p^{-1} + \ldots + a_{-K} p^{-K},$$

где коэффициенты a_i − символы в изображении числа R , которые принимают значения от 0 до $p - 1$. Обычно число $R_{(p)}$ представляется записью коэффициентов a_i: $R_{(p)} = a_N\, a_{N-1} \ldots a_1\, a_0\ \ a_{-1} \ldots a_{-K}$. Например:

$$265{,}23_{(10)} = 2 \cdot 10^2 + 6 \cdot 10^1 + 5 \cdot 10^0 + 2 \cdot 10^{-1} + 3 \cdot 10^{-2},$$
$$23{,}78_{(16)} = 2 \cdot 16^1 + 3 \cdot 16^0 + 7 \cdot 16^{-1} + 8 \cdot 16^{-2},$$
$$10001{,}01_{(2)} = 1 \cdot 2^4 + 0 \cdot 2^3 + 0 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 + 0 \cdot 2^{-1} + 1 \cdot 2^{-2}.$$

Никаких принципиальных отличий в арифметических действиях в системах счисления, отличных от десятичной, нет. Необходимо преодолеть лишь небольшой психологический барьер, и двоичная, восьмеричная и шестнадцатеричная арифметики покорятся навеки.

3.1. Арифметика в двоичной системе счисления

В двоичной системе счисления используются две – цифры 0 и 1. Основание двоичной системы счисления записывается в виде

$$\mathbf{2_{(10)} = 2 \cdot 10^0 = 1 \cdot 2^1 + 0 \cdot 2^0 = 10_{(2)}.}$$

Двоичная таблица сложения

+	0	1
0	0	1
1	1	10

Двоичная таблица умножения

*	0	1
0	0	0
1	0	1

Сложение

Следующие простые правила иллюстрируют операцию сложения положительных целых чисел в двоичной системе счисления:

```
  0     0     1
+     +     +
  0     1     1
-----------------
  0     1    10
```

В последнем правиле произошло увеличение разрядности суммы по сравнению со слагаемыми на 1 бит. Такой бит называют ***битом переноса*** (***carry bit***). Пусть требуется сложить два положительных целых числа в двоичной системе счисления:

```
    1 1 1 1 1 1 1 1 0 0 0 0 1 0 1 0 1 1 1 0 1
+   1 0 0 1 1 0 0 0 1 1 1 1 1 1 0 0 0 1 0 1 0
-----------------------------------------------
  1 1 0 0 1 1 0 0 0 0 0 0 0 0 1 1 1 0 0 1 1 1
```

Вычитание

Аналогичные правила действуют и для операции вычитания:

```
    1 1 1 1 1 1 1 1 0 0 0 0 1 0 1 0 1 1 1 0 1
-   1 0 0 1 1 0 0 0 1 1 1 1 1 1 0 0 0 1 0 1 0
-----------------------------------------------
      1 1 0 0 1 1 0 0 0 0 0 1 1 1 0 1 0 0 1 1
```

Умножение

Операцию умножения можно производить привычным способом в столбик:

```
                  *  1 1 1 0 1
                     1 0 0 1 0
                    ---------
                     0 0 0 0 0
                   1 1 1 0 1
                 0 0 0 0 0
               0 0 0 0 0
             1 1 1 0 1
           ---------------
           1 0 0 0 0 0 1 0 1 0
```

Приведем некоторые ***примеры*** выполнения основных операций в двоичной системе счисления.

а) сложение:

$$\begin{array}{r} 0{,}000011101 \\ +\ 0{,}110000011 \\ \hline 0{,}110100000 \end{array}$$

б) вычитание:

$$\begin{array}{r} 1{,}110010101 \\ -\ 0{,}100001011 \\ \hline 1{,}010001010 \end{array}$$

в) умножение:

$$\begin{array}{r} 1{,}1001 \\ \times\ \ 0{,}11 \\ \hline 11001 \\ 11001 \\ \hline 1{,}001011 \end{array}$$

г) деление:

$$\begin{array}{l|l} 111{,}10101 & 1{,}01 \\ \underline{101} & \overline{110{,}001} \\ \ 101 & \\ \underline{101} & \\ \ \ 0000101 & \\ \ \ \ \ \underline{101} & \\ \ \ \ \ \ \ 0 & \end{array}$$

3.2. Арифметика в восьмеричной системе счисления

В восьмеричной системе счисления используются цифры 0, 1, 2, 3, 4, 5, 6, 7. Основание записывается в виде

$$\mathbf{8_{(10)} = 8 \cdot 10^0 = 1 \cdot 8^1 + 0\ 8^0 = 10_{(8)}.}$$

Восьмеричная таблица сложения

+	1	2	3	4	5	6	7
1	2	3	4	5	6	7	10
2	3	4	5	6	7	10	11
3	4	5	6	7	10	11	12
4	5	6	7	10	11	12	13
5	6	7	10	11	12	13	14
6	7	10	11	12	13	14	15
7	10	11	12	13	14	15	16

Восьмеричная таблица умножения

*	1	2	3	4	5	6	7
1	1	2	3	4	5	6	7
2	2	4	6	10	12	14	16
3	3	6	11	14	17	22	25
4	4	10	14	20	24	30	34
5	5	12	17	24	31	36	43
6	6	14	22	30	36	44	52
7	7	16	25	34	43	52	61

Приведем некоторые ***примеры*** выполнения основных операций в восьмеричной системе счисления.

а) сложение:

$$\begin{array}{r} 54{,}724 \\ +\ 34{,}731 \\ \hline 111{,}655 \end{array}$$

б) вычитание:

$$\begin{array}{r} 1547{,}214 \\ -\ \ 54{,}331 \\ \hline 1472{,}663 \end{array}$$

в) умножение:

$$\begin{array}{r} 15{,}21 \\ \times\ \ 4{,}31 \\ \hline 1521 \\ 4763 \\ 6504 \\ \hline 72{,}1751 \end{array}$$

г) деление:

$$\begin{array}{r|l} 451{,}67 & 321 \\ \hline 321 & 1{,}331\ldots \\ 1306 & \\ 1163 & \\ \hline 1237 & \\ 1163 & \\ \hline 00540 & \\ 321 & \\ \hline 217\ldots & \end{array}$$

3.3. Арифметика в шестнадцатеричной системе счисления

В шестнадцатеричной системе счисления используются цифры 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 и буквы A, B, C, D, E, F. Основание шестнадцатеричной системы записывается в виде

$$\mathbf{16_{(10)}=16\cdot10^0=1\cdot16^1+0\cdot16^0=10_{(16)}.}$$

Приведем некоторые ***примеры*** выполнения основных операций в шестнадцатеричной системе счисления.

а) сложение:

```
 197,AB
+ 54,42
--------
 1EB,ED
```

б) вычитание:

```
 1A35,F1
-  1EB,ED
----------
 184A,04
```

в) умножение:

```
    37,21
  ×    54
----------
    DC84
  113A5
----------
  1216,D4
```

г) деление:

```
82,B1 | 43
43    | 1,F3...
-----
3FB
3ED
---
 E1
 C9
---
 18...
```

Шестнадцатеричная таблица сложения

+	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	10
2	3	4	5	6	7	8	9	A	B	C	D	E	F	10	11
3	4	5	6	7	8	9	A	B	C	D	E	F	10	11	12
4	5	6	7	8	9	A	B	C	D	E	F	10	11	12	13
5	6	7	8	9	A	B	C	D	E	F	10	11	12	13	14
6	7	8	9	A	B	C	D	E	F	10	11	12	13	14	15
7	8	9	A	B	C	D	E	F	10	11	12	13	14	15	16
8	9	A	B	C	D	E	F	10	11	12	13	14	15	16	17
9	A	B	C	D	E	F	10	11	12	13	14	15	16	17	18
A	B	C	D	E	F	10	11	12	13	14	15	16	17	18	19
B	C	D	E	F	10	11	12	13	14	15	16	17	18	19	1A
C	D	E	F	10	11	12	13	14	15	16	17	18	19	1A	1B
D	E	F	10	11	12	13	14	15	16	17	18	19	1A	1B	1C
E	F	10	11	12	13	14	15	16	17	18	19	1A	1B	1C	1D
F	10	11	12	13	14	15	16	17	18	19	1A	1B	1C	1D	1E

Шестнадцатеричная таблица умножения

×	1	2	3	4	5	6	7	8	**9**	A	B	C	D	E	F
1	1	2	3	4	5	6	7	8	**9**	A	B	C	D	E	F
2	2	4	6	8	A	C	E	10	12	14	16	18	1A	1C	1E
3	3	6	9	C	F	12	15	18	1B	1E	21	24	27	2A	2D
4	4	8	C	10	14	18	1C	20	24	28	2C	30	34	38	3C
5	5	A	F	14	19	1E	23	28	2D	32	37	3C	41	46	4B
6	6	C	12	18	1E	24	2A	30	36	3C	42	48	4E	54	5A
7	7	E	15	1C	23	2A	31	38	3F	46	4D	54	5B	62	69
8	8	10	18	20	28	30	38	40	48	50	58	60	68	70	78
9	9	12	1B	24	2D	36	3F	48	51	5A	63	6C	75	7E	87
A	A	14	1E	28	32	3C	46	50	5A	64	6E	78	82	8C	96
B	B	16	21	2C	37	42	4D	58	63	6E	79	84	8F	9A	A5
C	C	18	24	30	3C	48	54	60	6C	78	84	90	9C	A8	B4
D	D	1A	27	34	41	4E	5B	68	75	82	8F	9C	A9	B6	C3
E	E	1C	2A	38	46	54	62	70	7E	8C	9A	A8	B6	C4	D2
F	F	1E	1E	2D	3C	4B	5A	69	78	87	A5	B4	C3	D2	E1

ПРИЛОЖЕНИЕ

Таблица степеней оснований основных систем счисления

Степень основания	2	8	16
0	1	1	1
1	2	8	16
2	4	64	256
3	8	512	4096
4	16	4096	65536
5	32	32768	1048576
6	64	262144	16777216
7	128	2097152	268435456
8	256	16777216	4294967296
9	512	134217728	68719476736
10	1024	1073741824	1099511627776
11	2048	8589934552	17592186044416
12	4096	68719476736	281474976710656
13	8192	549755813888	4503599627370496
14	16384	4398046511104	72057594037927936
15	32768	35184372088832	1152921504606846976
16	65536	281474976710756	18446744073709551616

Таблица соответствия первых 17 положительных чисел основных систем счисления

Двоичная система	Восьмеричная система	Десятичная система	Шестнадцатеричная система
0	0	0	0
1	1	1	1
10	2	2	2
11	3	3	3
100	4	4	4
101	5	5	5
110	6	6	6
111	7	7	7
1000	10	8	8
1001	11	9	9
1010	12	10	A
1011	13	11	B
1100	14	12	C
1101	15	13	D
1110	16	14	E
1111	17	15	F
10000	20	16	10

Практические занятия

Тема 1: Системы счисления. Арифметические операции в разных системах счисления. Перевод из одной системы счисления в другую

Варианты заданий

1. Представить следующие числа в развернутом виде:

а) $54{,}987_{(10)}$;	б) $1011{,}011_{(2)}$;	в) $DE{,}7_{(16)}$;
г) $A4{,}187_{(16)}$;	д) $14{,}008_{(10)}$;	е) $0{,}00011_{(2)}$;
ж) $3214{,}47_{(8)}$;	з) $555{,}654_{(8)}$;	и) $123{,}0111_{(16)}$.

2. Выполнить указанные арифметические операции над числами в соответствующих системах счисления:
   1) в двоичной системе счисления:
      а) $10011{,}1 + 11{,}00111$; б) $1111{,}0111 - 1{,}0001$;
      в) $111{,}01 \cdot 1{,}01$; г) $\dfrac{1001{,}11}{11{,}01}$.
   2) в восьмеричной системе счисления:
      а) $34{,}1 + 11{,}17$; б) $12{,}121 - 1{,}1755$;
      в) $62{,}1 \cdot 67{,}17$; г) $\dfrac{174{,}23}{34{,}5}$.
   3) в шестнадцатеричной системе счисления:
      а) $A23{,}F1 + 1{,}7$; б) $1343{,}31 - D1{,}7F$;
      в) $23{,}F1 \cdot A{,}7$; г) $\dfrac{231{,}CD}{1{,}67}$.
3. Перевести следующие числа из двоичной системы счисления в восьмеричную и шестнадцатеричную системы счисления:
   а) 111000111,11101; б) -111,101;
   в) 111011,101; г) -10000111011,101;
   д) -1000111,1110111101; е) -11011,10111.

4. Перевести следующие числа из двоичной системы счисления в десятичную (в дробной части получить четыре знака после запятой):
   а) 1100001,111; б) 11001,1011;
   в) −1,01110111; г) 1001,0001.
5. Перевести следующие числа из шестнадцатеричной системы счисления в десятичную (в дробной части получить четыре знака после запятой):
   а) 1AF01,21; б) 1931,C1;
   в) −1,41A; г) 0,00A6.
6. Перевести следующие числа из шестнадцатеричной системы счисления в двоичную:
   а) A01C,1FF; б) -31,F104;
   в) −B1,1001; г) 68,01FA.
7. Перевести следующие числа из десятичной системы счисления в шестнадцатеричную (в дробной части получить четыре знака после запятой):
   а) 784,19; б) -31,04;
   в) −121,1001; г) -68,976.

Практические занятия

Тема 2: Основы алгоритмизации. Построение блок-схем линейных и разветвляющихся вычислительных процессов

Теоретические сведения. Решение любой задачи на ЭВМ происходит в несколько этапов: постановка задачи; конструирование алгоритма решения задачи; составление программы по разработанному алгоритму; ввод в ВМ программы и исходных данных; отладка и тестирование программы; получение решения и анализ результатов.

Алгоритм – конечная последовательность точно определенных действий, приводящих к однозначному решению поставленной задачи. Главная особенность любого алгоритма – *формальное исполнение*, позволяющее выполнять заданные действия (команды) не только человеку, но и различным техническим устройствам (исполнителям). Процесс составления алгоритма называется *алгоритмизацией.* Алгоритмы могут быть заданы: словесно, таблично, графически (с помощью блок-схем). *Словесное* задание описывает алгоритм с помощью слов и предложений. *Табличное* задание служит для представления алгоритма в форме таблиц и расчетных формул. *Графическое* задание, или *блок-схема,* – способ представления алгоритма с помощью геометрических фигур, называемых *блоками.* Последовательность блоков и соединительных линий образуют блок-схему. Описание алгоритмов с помощью блок-схем является наиболее наглядным и распространенным способом задания алгоритмов. Блок-схемы располагаются сверху вниз. Линии соединения отдельных блоков показывают направление процесса обработки в схеме. Каждое такое направление называется ветвью. Алгоритм независимо от его структуры всегда имеет по одному блоку «Начало» и «Конец». Его ветви должны в конце сойтись, и по какой бы ветви не было бы начато движение, оно всегда должно привести к блоку «Конец».

При задании алгоритма с помощью блок-схемы используются строго определенные блоки. Основные типы блоков приведены в таблице 1.1. Следует отметить, что все блоки нумеруются. В этом случае номера проставляются вверху слева от блока (блоки «Начало», «Конец» и соединительные блоки не

нумеруются). Стрелки на соединяющих линиях обычно не ставят при направлении сверху вниз и слева направо; если направление противоположное, то его показывают стрелкой на линии.

Таблица 1.1. Основные типы блоков

	*Начало и конец алгоритма* (для функций «Вход», «Выход»)
	*Блок обработки*. Внутри блока записываются формулы, обозначения и функции
	*Блок условия*. Внутри блока записываются условия выбора направления действия алгоритма
	*Блок предопределенного процесса* (функция/ подпрограмма)
	*Блок ввода информации*
	*Блок цикла с известным количеством повторений*
	*Соединительный блок*
	*Блок вывода информации на печатающее устройство*

Алгоритмы бывают *линейные, разветвляющиеся* и *циклические*.

Линейный алгоритм не содержит логических условий, имеет одну ветвь обработки и изображается линейной последовательностью связанных друг с

другом блоков. Условное изображение линейного алгоритма может быть представлено на рис. 1.1.

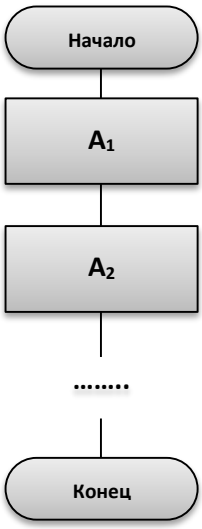


Рис. 1.1. Условное изображение линейного алгоритма

Пример 1.1. Составить блок-схему вычисления $z = \varphi(y)$, $y = f(x)$, где f, φ − известные функции, при заданном значении переменной x (рис. 1.2).

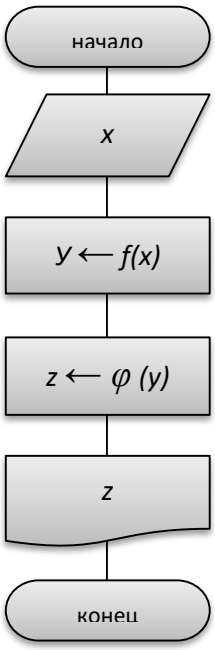


Рис. 1.2. Блок-схема примера 1.1

Знак «←» означает: внести значение переменной в ячейку памяти с определенным именем (операция присваивания).

Разветвляющийся алгоритм содержит одно или несколько логических условий и имеет несколько ветвей обработки (пример 1.2). Условное изображение разветвления представлено на рис. 1.3.

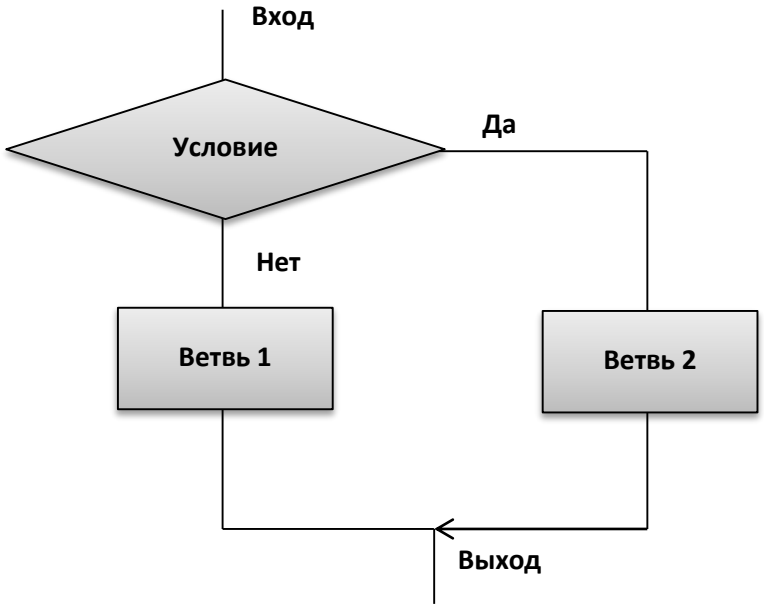


Рис. 1.3. Условное изображение разветвляющегося алгоритма

Пример 1.2. Определить попадает ли точка внутрь круга, если радиус круга известен, а его центр и точка задаются своими координатами (рис. 1.4).

Принятые обозначения:

- x, y – координаты точки
- $x_ц$, $y_ц$ – координаты центра круга
- R – радиус круга

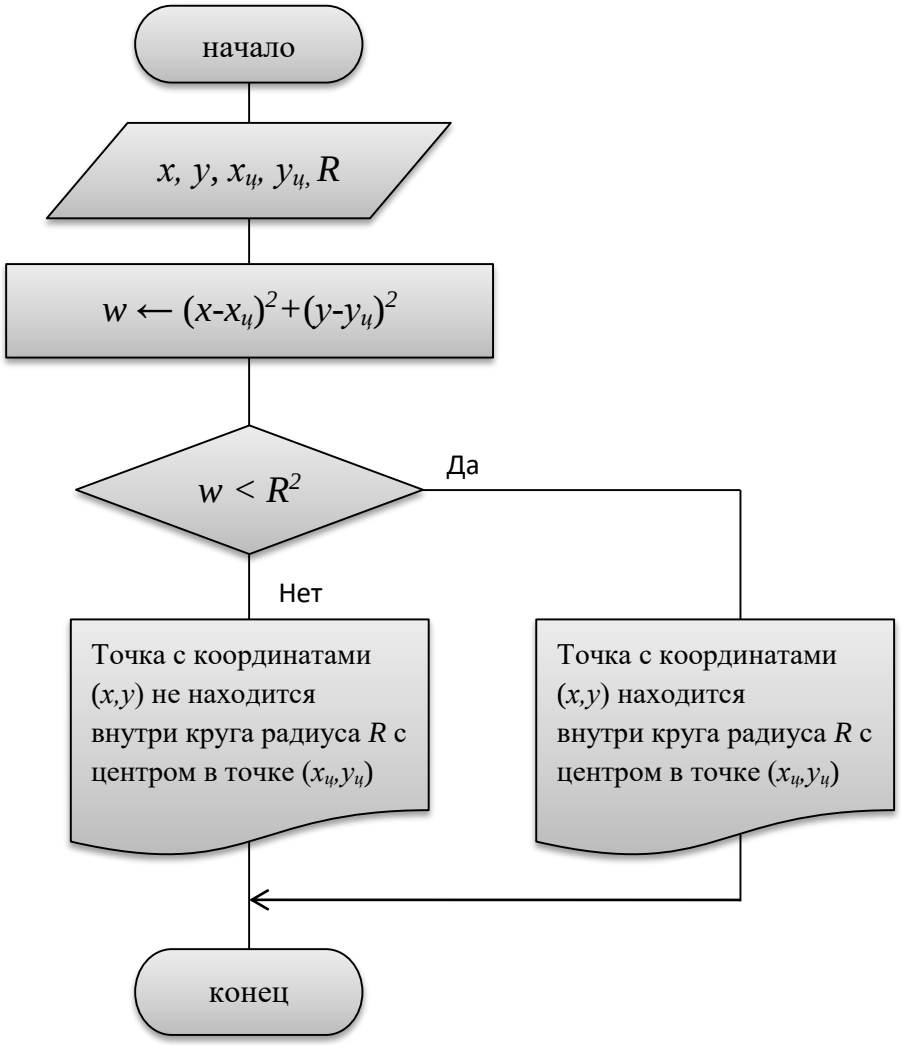


Рис. 1.4. Блок-схема примера 1.2

Пример 1.3. Составить блок-схему нахождения корней квадратного уравнения.

Решаем уравнение $ax^2+bx+c=0$. Известно, что действительные корни квадратного уравнения определяются по формуле

$$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}, \quad D = b^2 - 4ac \geq 0.$$

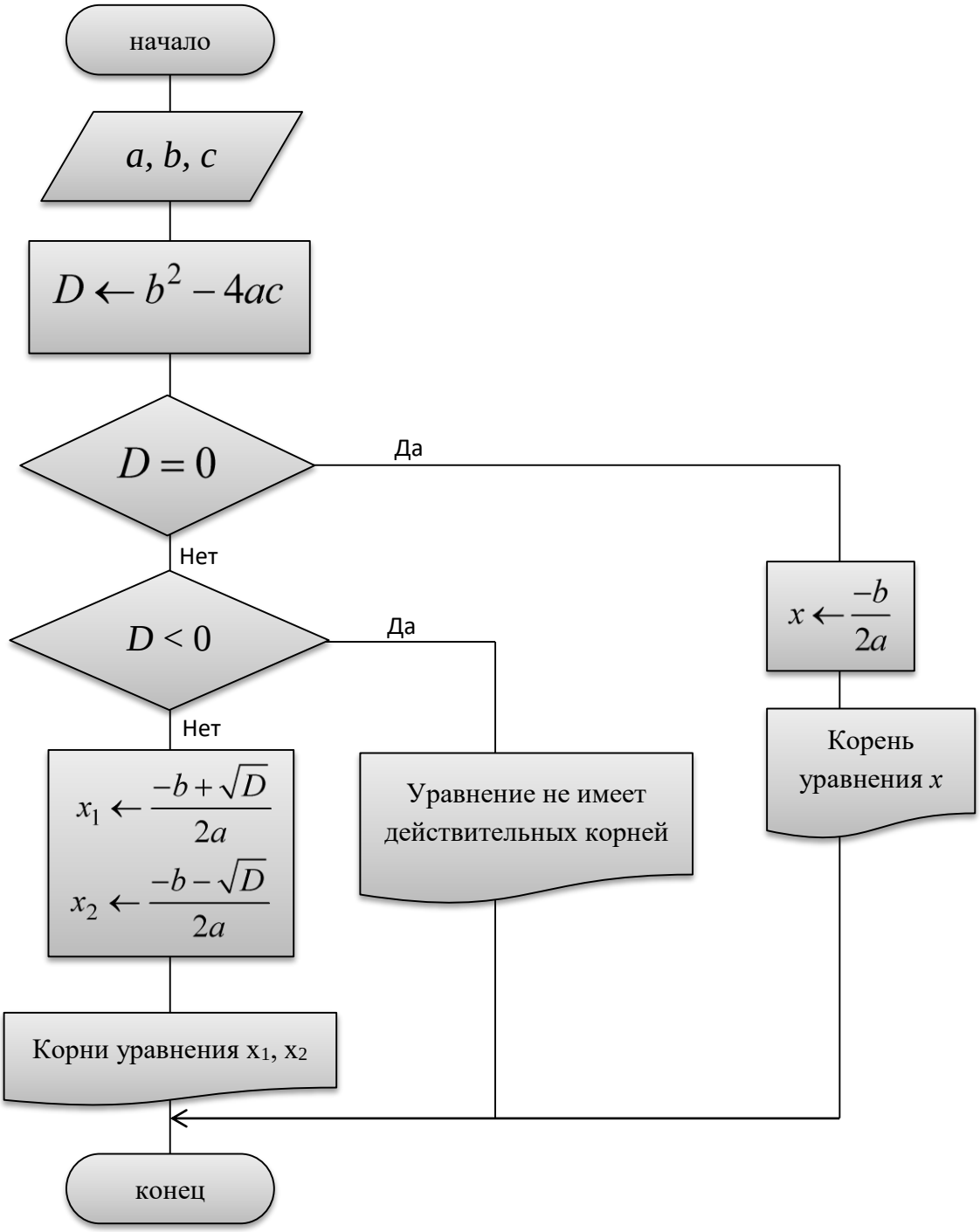


Рис. 3.5. Блок-схема примера 1.3

Варианты заданий

1. Даны x, y. Получить $\dfrac{|x|-|y|}{1+|xy|}$.
2. Даны два числа. Найти среднее арифметическое кубов этих чисел и среднее геометрическое модулей этих чисел.
3. Вычислить расстояние между двумя точками с данными координатами.
4. Найти площадь треугольника со сторонами a, b, c по формуле Герона: $S=\sqrt{p(p-a)(p-b)(p-c)}$, где $p=\dfrac{(a+b+c)}{2}$.
5. Даны x, y, z. Найти w, если:

а) $w=\sin\left|\left(y-\sqrt{|x|}\right)\left(x-\dfrac{y}{z^2+\dfrac{x^2}{4}}\right)\right|$; б) $w=x-\dfrac{x^2}{1+\sin^2(x+y+z)}$;

в) $w=\cos\left(z^2+\dfrac{x^2}{4}+y\right)$; г) $w=y+x-\dfrac{\cos^2 x}{1+\sqrt{|y+z|}}$.

6. Даны два числа. Вывести первое число, если оно больше второго, и оба числа, если это не так.
7. Найти наименьшее из трех данных чисел.
8. Даны три числа, являющиеся длинами сторон треугольника. Определить тип треугольника (равносторонний, равнобедренный, разносторонний).
9. Даны x, y, z. Найти:

а) $\max\{x+y+z, xyz\}+3$; б) $\min\{x^2+y^2, y^2+z^2\}-4$.

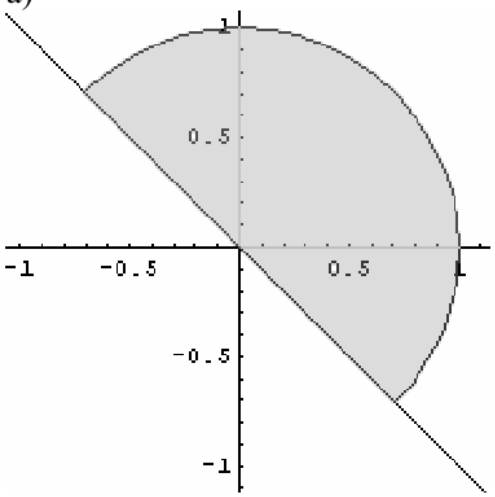
10. Дано x. Вычислить y, если:

а) $y=\begin{cases}x^2, \text{при } -2\le x\le 2,\\ 4, \text{при } x<-2 \text{ и } x>2\end{cases}$; б) $y=\begin{cases}0, \text{при } x\le 0\\ x, \text{при } 0<x\le 1,\\ x^4, \text{при } x>1\end{cases}$;

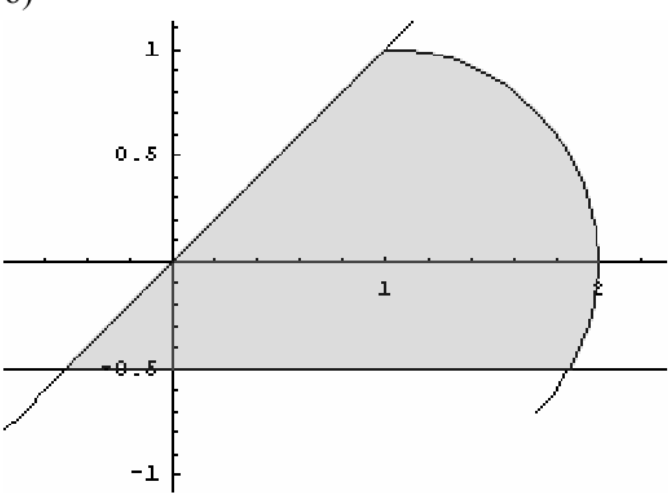
в) $y=\begin{cases}x^2+4x+5, \text{при } x\le 2,\\ \dfrac{1}{x^2+4x+5}, \text{при } x>2\end{cases}$; г) $y=\begin{cases}0, \text{при } x\le 0\\ x^2-x, \text{при } 0<x\le 1,\\ x^2-\sin\pi x^2-1, \text{при } x>1\end{cases}$.

11. Определить попадает ли заданная точка внутрь заданной области.

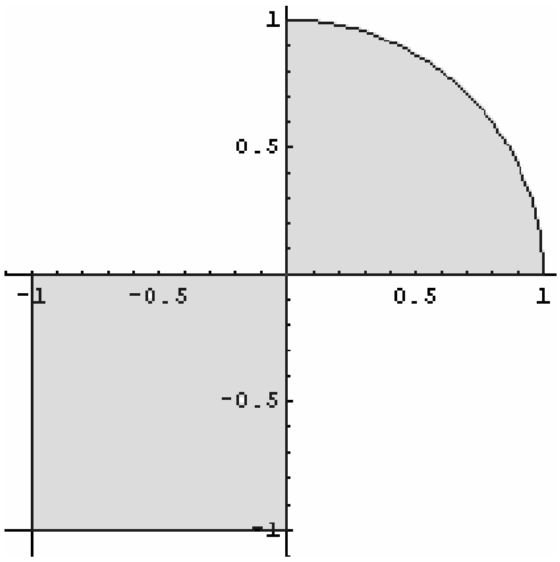
а)



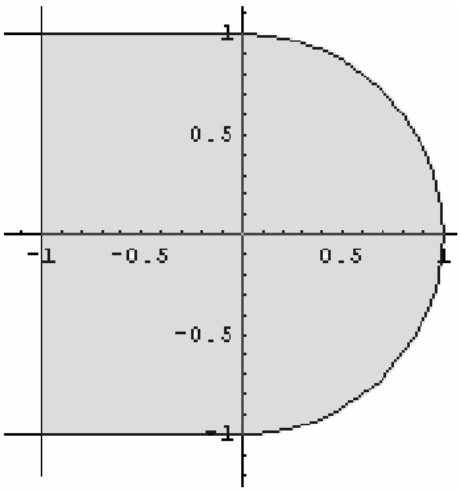
б)



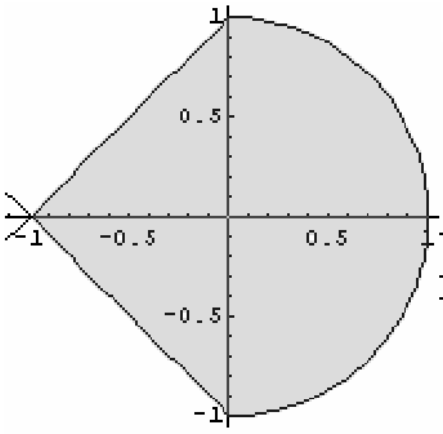
в)



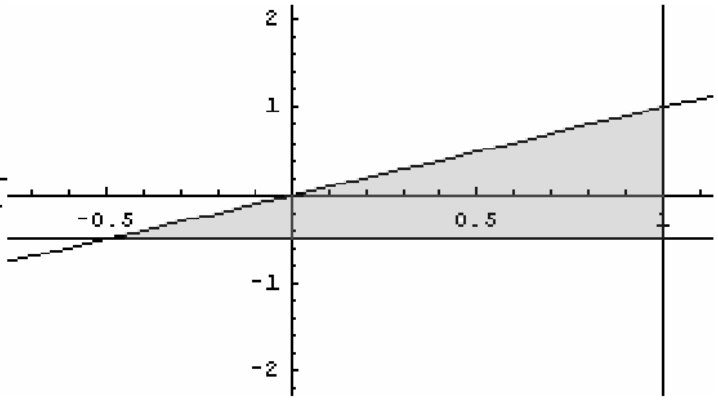
г)



д)



е)



Практические занятия

Тема 3: Построение блок-схем циклических вычислительных процессов

Теоретические сведения. *Циклический* алгоритм содержит один или несколько циклов. *Цикл* – это многократно повторяемая часть алгоритма. Цикл, не содержащий внутри себя других циклов, называют простым. Если он содержит внутри себя другие циклы или разветвления, то цикл называют сложным или вложенным. Любой цикл характеризуется одной или несколькими переменными, называемыми параметрами цикла, от анализа значений которых зависит выполнение цикла. *Параметр цикла* – переменная, принимающая при каждом вхождении в цикл новое значение. Условное изображение циклического алгоритма представлено на рис. 2.1.

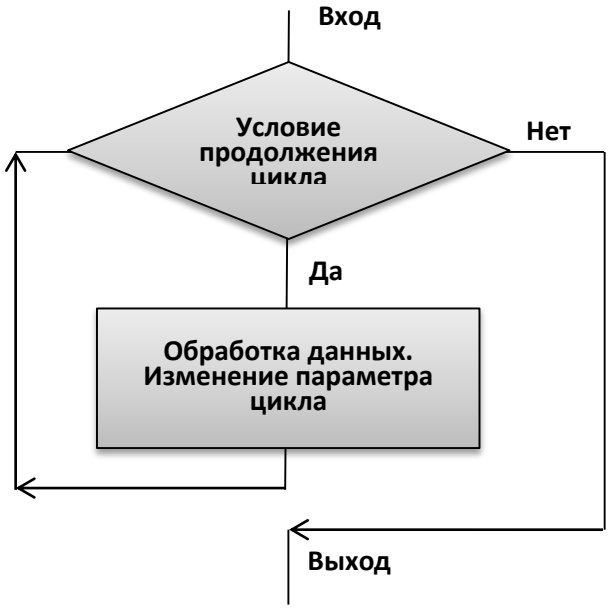


Рис. 2.1. Условное изображение циклического алгоритма

Пример 2.1. Составить блок-схему нахождения суммы N первых целых чисел от 0 до N −1 (рис. 2.2, рис. 2.3).

Вариант 1.

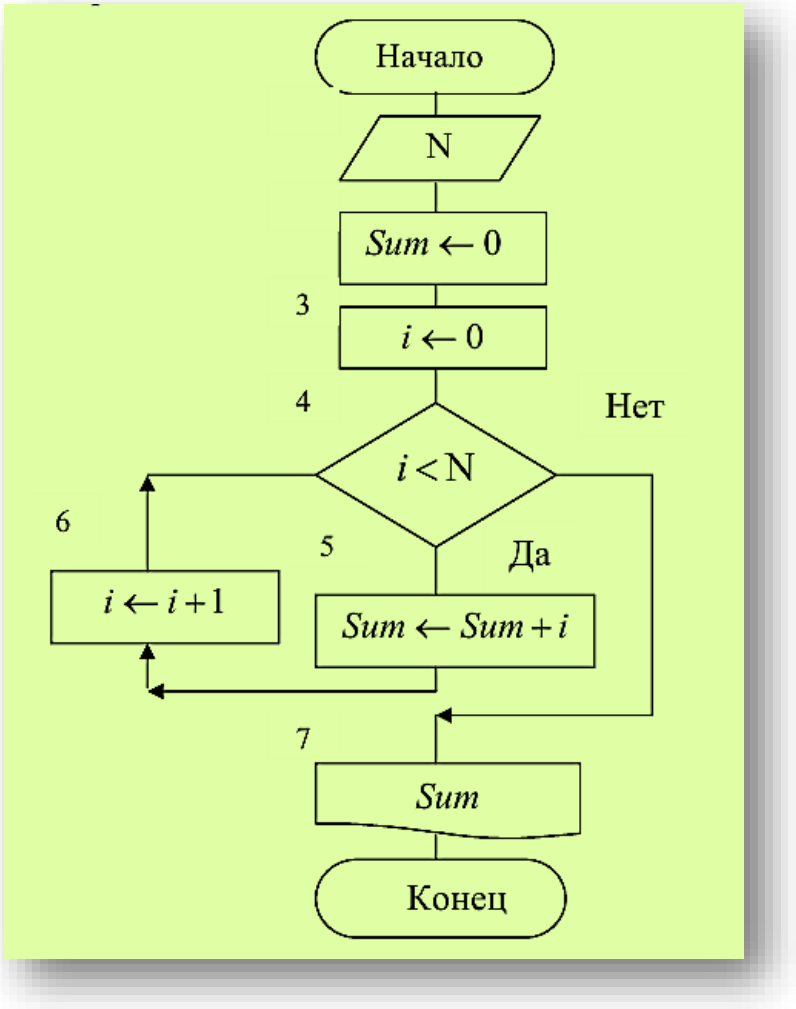


Рис. 2.2. Блок-схема примера 2.1

Блок-схема любого циклического процесса (пример 2.1, вариант 1) независимо от многообразия сводящихся к нему задач должна содержать

- блок задания начального значения параметра цикла (блок 3)
- блок реализации необходимых вычислений (блок 5)
- блок изменения параметра цикла (блок 6)
- блок проверки условия окончания цикла (блок 4).

Способ организации цикла зависит от условия задачи. Во многих задачах количество повторений цикла указывается (пример 2.1, вариант 2).

Это так называемые циклы с известным количеством повторений или циклы со счетчиком.

Пример 2.1. *Вариант 2.*

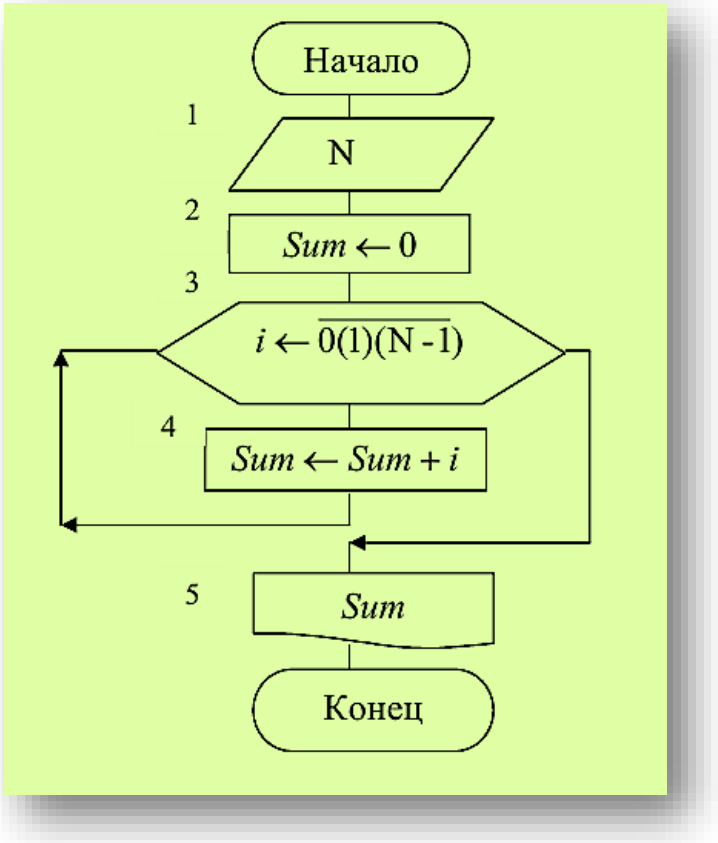


Рис. 2.3. Блок-схема примера 2.1

Во многих задачах встречаются так называемые *циклы по простой переменной.* В этом случае задается только начальное значение переменной, закон (правило), по которому она изменяется, и конечное ее значение. В процессе решения задачи текущее значение переменной каждый раз сравнивается с е конечным значением. Выход из цикла происходит, когда переменная достигла своего конечного значения или впервые превысила его. В таких циклах количество повторений неизвестно, но оно может быть вычислено, поскольку значение переменной изменяется по некоторому закону (пример 2.2).

Пример 2.2. Составить блок-схему нахождения значений функции *f(x) = sin x* для значений переменной *x*, изменяющейся в диапазоне от *a* до *b* шагом *h* (рис. 2.4).

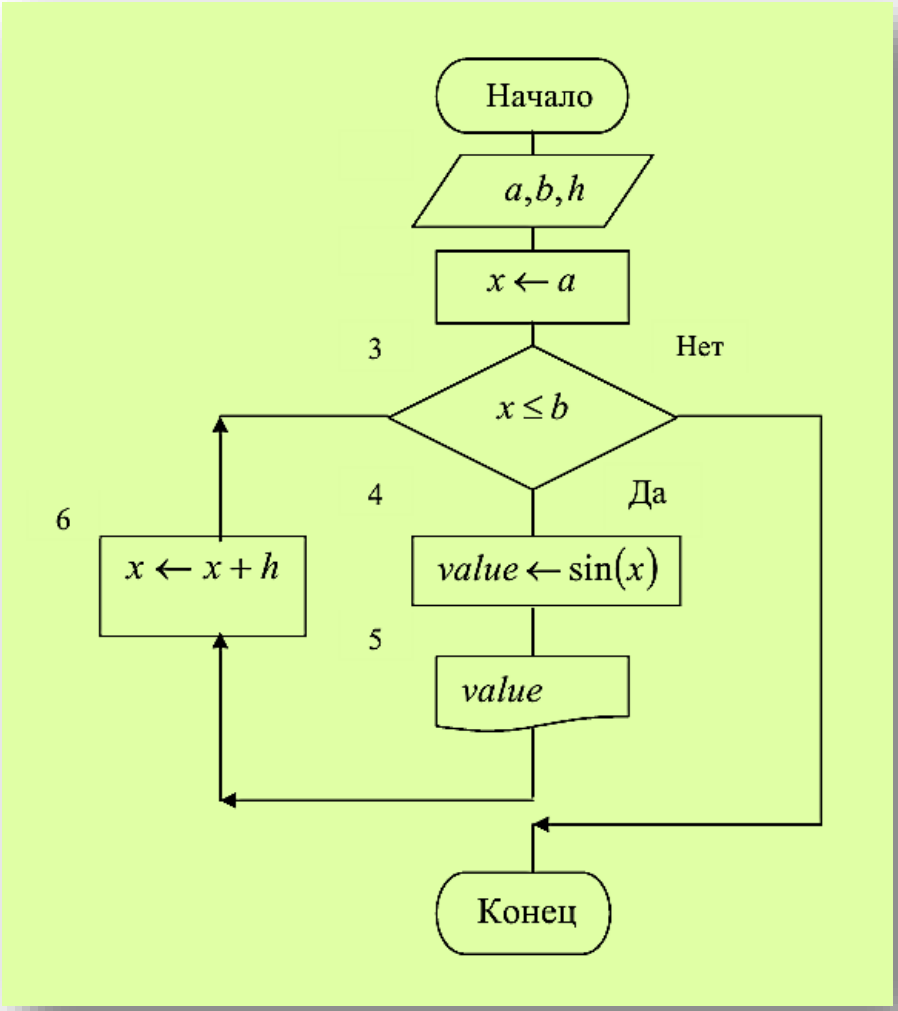


Рис. 2.4. Блок-схема примера 2.2

Пример 2.3. Составить блок-схему вычисления суммы вида (рис. 2.5).

$$x - \frac{x^3}{3!} + \frac{x^5}{5!} - \ldots + (-1)^{m-1} \frac{x^{2m-1}}{(2m-1)!} \quad x \in R,\ m \in N$$

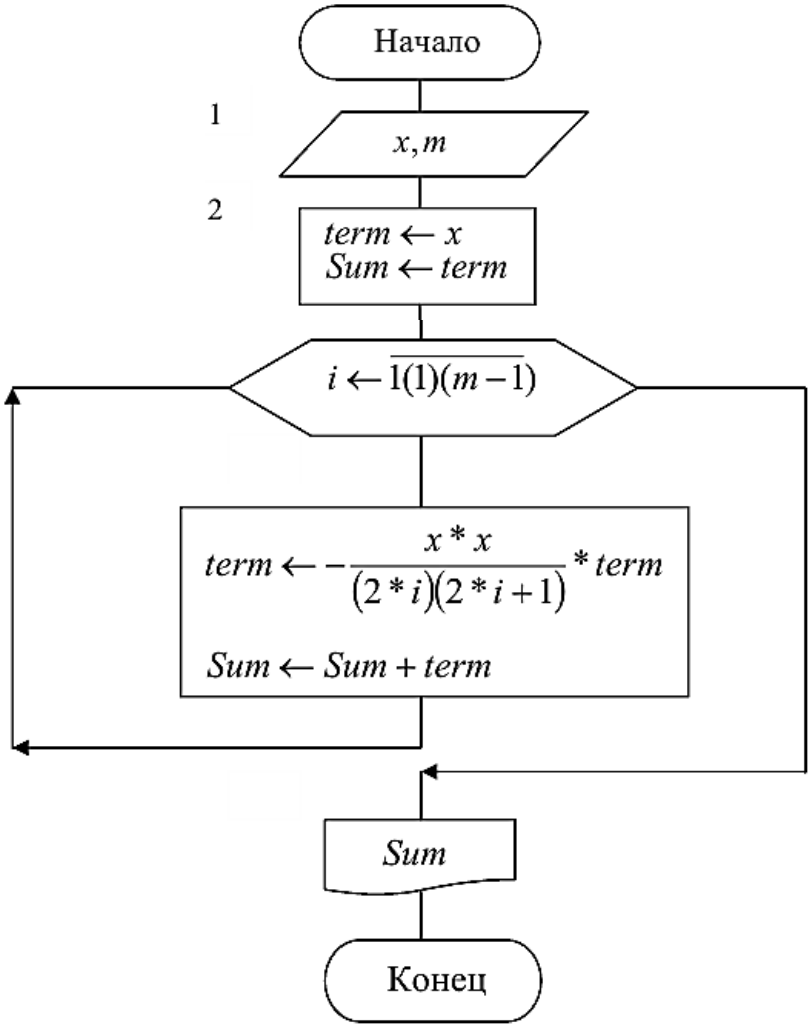


Рис. 2.5. Блок-схема примера 2.3.

Варианты заданий

1. Дано натуральное n. Вычислить

а) $\frac{1}{1}+\frac{1}{2}+\ldots+\frac{1}{n}$;

б) 2^n;

в) $\left(1+\frac{1}{1^2}\right)\left(1+\frac{1}{2^2}\right)\ldots\left(1+\frac{1}{n^2}\right)$;

г) $\frac{1}{1\cdot 2}-\frac{1}{2\cdot 3}+\ldots+\frac{(-1)^{n+1}}{n(n+1)}$;

д) $\frac{1}{1^5}+\frac{1}{2^5}+\ldots+\frac{1}{n^5}$;

е) $\frac{1}{3^2}+\frac{1}{5^2}+\ldots+\frac{1}{(2n+1)^2}$;

ж) $-\frac{1}{3}+\frac{1}{5}+\ldots+\frac{(-1)^n}{2n+1}$;

з) $\frac{1!}{1}+\frac{2!}{1+\frac{1}{2}}+\ldots+\frac{n!}{1+\frac{1}{2}+\ldots+\frac{1}{n}}$;

и) $\underbrace{\sqrt{2+\sqrt{2+\ldots.\sqrt{2}}}}_{\text{n корней}}$;

к) $\frac{1}{\sin 1}+\frac{1}{\sin 1+\sin 2}+\ldots+\frac{1}{\sin 1+\sin 2+\ldots+\sin n}$.

2. Дано натуральное n и действительное x. Вычислить

№	$\sum$
1	$1+\frac{x}{1!}+\frac{x^2}{2!}+\ldots+\frac{x^n}{n!}$
2	$\left(\frac{1}{1!}+\sqrt{\lvert x\rvert}\right)+\left(\frac{1}{2!}+\sqrt{\lvert x\rvert}\right)+\ldots+\left(\frac{1}{n!}+\sqrt{\lvert x\rvert}\right)$
3	$\left(1+\frac{\sin x}{1!}\right)\left(1+\frac{\sin 2x}{2!}\right)\ldots\left(1+\frac{\sin nx}{n!}\right)$
4	$x-\frac{x^3}{3!}+\frac{x^5}{5!}-\ldots+(-1)^{m-1}\frac{x^{2m-1}}{(2m-1)!}$
5	$1-\frac{x^2}{2!}+\frac{x^4}{4!}-\ldots+(-1)^m\frac{x^{2m}}{(2m)!}$
6	$x+\frac{x^3}{3!}+\frac{x^5}{5!}+\ldots+\frac{x^{2m-1}}{(2m-1)!}$
7	$1+\frac{x^2}{2!}+\frac{x^4}{4!}+\ldots+\frac{x^{2m}}{(2m)!}$

8	$x-\frac{x^2}{2}+\frac{x^3}{3}+\ldots+(-1)^{n-1}\frac{x^n}{n}$
9	$x-\frac{x^3}{3}+\frac{x^5}{5}+\ldots+(-1)^{m-1}\frac{x^{2m-1}}{2m-1}$
10	$-x-\frac{x^2}{2}-\frac{x^3}{3}-\ldots-\frac{x^n}{n}$

3. Вычислить значения функции $f(x)$ на отрезке $[a;b]$ с шагом h.

№	$f(x)$	$[a,b]$	h
1	2	3	4
1	$\ln(x)$	1;1.5	0.1
2	$1+\ln^2(x)$	0.4;1	0.1
3	$1+e^x$	0.5;0.6	0.01
4	$e^{x^2}/2$	2;3	0.2
5	$\cos(x)e^{-x}$	1;2	0.2
6	$1/(1+e^{-x})$	3;4	0.2
7	$\sin(x)\mathrm{sh}(x)$	1;5	1
8	$0.5+\mathrm{sh}^2(x)$	2;3	0.2
9	$\sqrt{x}\mathrm{ch}(x)$	3;4	0.2
10	$1/(1+\mathrm{ch}^2(x))$	2;4	0.5
11	$\sqrt{x}\mathrm{sh}(x)$	1;5	1
12	$e^{-x}\mathrm{ch}(x)$	1;4	1
13	$\ln(x^2)$	1;1.4	0.1
14	$x+\ln(x)$	1;5	1
15	$1/(1+\sin x)$	$\pi/3;\pi/6$	$\pi/10$
16	$\sin x+\sqrt{x}$	$\pi/6;\pi/4$	$\pi/10$
17	$x(1-\cos x)$	0.4;0.8	0.1
18	$e^{x+3}\sin x$	0;2	0.5
19	$\cos(x)\mathrm{ch}(x)$	1;5	1
20	$e^{1+x}\mathrm{sh}(x)$	1;4	1

Практические занятия

Тема 4: Построение блок-схем итерационных вычислительных процессов

Теоретические сведения. Существует ряд циклических вычислительных процессов, называемых *итерационными*. Итерационный процесс продолжается до тех пор, пока разность между соседними, уточняемыми на каждом шаге цикла значениями (итерациями), не окажется меньше либо равной некоторой заданной величине (точности). Характерной особенностью итерационного процесса является то, что количество циклов (итераций) в нем заранее не известно и становится определенным только после окончания вычислений. Выход из цикла происходит тогда, когда анализируемая величина или величины на очередной итерации отличаются от эталонных величин (например, заданной точности). В таком цикле, как правило, результаты вычислений предыдущего шага цикла используются как исходные данные при выполнении следующего шага цикла (пример 3.1).

Пример 3.1. Составить блок-схему нахождения приближенного значения квадратного корня $y = \sqrt{x}$, $x > 0$ как предела последовательности

$$y_1, y_2, \ldots, y_k, \ldots, \quad \text{где} \quad y_{k+1} = \frac{1}{2}\left(y_k + \frac{x}{y_k}\right)$$

Точность полученного приближения оценить по величине отклонения двух соседних приближений y_k и y_{k+1}, $\Delta_{k+1} = |y_k - y_{k+1}| \le \varepsilon$, где $\varepsilon > 0$ - заданная точность вычислений (рис. 3.1).

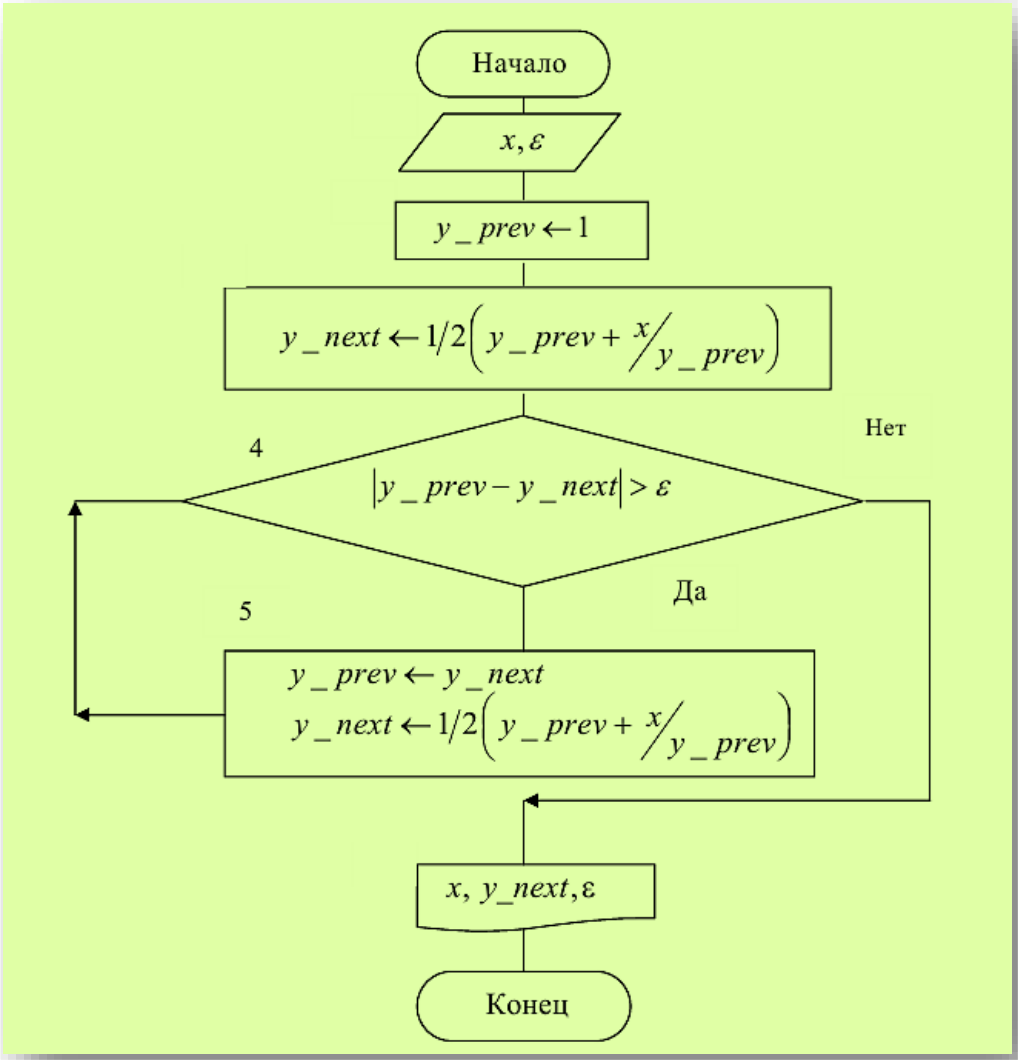


Рис. 3.1. Блок-схема примера 3.1

Пример 3.2. Составить блок-схему вычисления с точностью ε > 0 отрезка степенного ряда для функции

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \ldots + (-1)^{m-1}\frac{x^{2m-1}}{(2m-1)!} + \ldots, \quad x \in R$$

Точность полученного значения считать достигнутой, если последний член ряда не превосходит по абсолютной величине заданного ε (рис. 3.2).

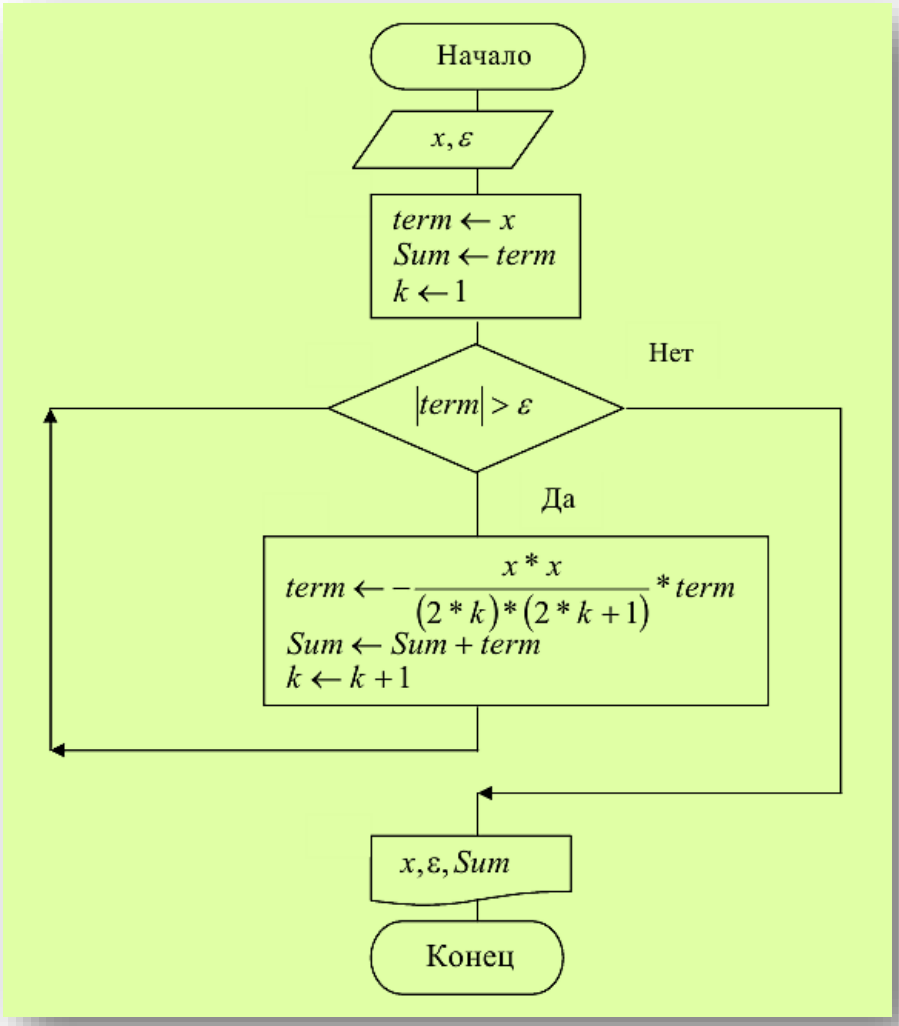


Рис. 3.2. Блок-схема примера 3.2

Варианты заданий

1. Даны действительные числа x, ε ($x \neq 0$, $\varepsilon > 0$). Вычислить приближенное значение бесконечной суммы. Вычисления выполнить с заданной точностью ε (пока текущий член ряда превосходит по абсолютной величине заданного ε).

а) $\sum_{k=0}^{\infty} \frac{(-1)^k x^{2k+1}}{k!(2k+1)}$;

б) $\sum_{k=0}^{\infty} \frac{(-1)^k x^{4k+3}}{(2k+1)!(4k+3)}$;

в) $\sum_{k=0}^{\infty} \frac{(-1)^k x^{4k+1}}{(2k)!(4k+1)}$;

г) $\sum_{k=0}^{\infty} \frac{(-1)^k}{((k+1)!)^2}\left(\frac{x}{2}\right)^{2(k+1)}$;

д) $\sum_{k=0}^{\infty} \frac{(-1)^k}{k!(k+1)!}\left(\frac{x}{2}\right)^{2k+1}$;

е) $\sum_{k=0}^{\infty} \frac{(-1)^{k+1}}{(2k)!}\left(\frac{x}{3}\right)^{4k}$;

ж) $\sum_{k=0}^{\infty} \frac{(-1)^{k+1} x^{2k-1}}{(2k-1)(2k+1)!}$;

з) $\sum_{k=0}^{\infty} \frac{x^{2k}}{2^k k!}$;

и) $\sum_{k=1}^{\infty} \frac{1}{x^3 k^2}$;

к) $\sum_{k=1}^{\infty} \frac{x^{2k}}{x^2 + k^3}$;

л) $\sum_{k=0}^{\infty} \frac{(-x)^{2k}}{2k!}$;

м) $\sum_{k=0}^{\infty} \frac{(-1)^k x^k}{(k+1)^2}$;

н) $\sum_{k=0}^{\infty} \frac{(-1)^k (k+1) x^k}{3^k}$;

о) $\sum_{k=0}^{\infty} \frac{(-1)^k x^{k+2}}{(k+1)(k+2)!}$;

п) $\sum_{k=1}^{\infty} \frac{(-1)^{k+1}}{(2k+1)!}\left(\frac{x}{3}\right)^{4k+2}$.

2. Даны действительные числа x, ε ($\varepsilon > 0$). Вычислить с заданной точностью ε приближенное значение бесконечной суммы и сравнить его с точным.

№	$\sum$	Точное значение
1	$1 + \frac{x}{1!} + \frac{x^2}{2!} + \ldots + \frac{x^n}{n!} + \ldots$	e^x, $x \in R$
2	$1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \ldots + (-1)^m \frac{x^{2m}}{(2m)!} + \ldots$	$\cos x$, $x \in R$
3	$x + \frac{x^3}{3!} + \frac{x^5}{5!} + \ldots + \frac{x^{2m-1}}{(2m-1)!} + \ldots$	$\mathrm{sh}x$, $x \in R$
4	$1 + \frac{x^2}{2!} + \frac{x^4}{4!} + \ldots + \frac{x^{2m}}{(2m)!} + \ldots$	$\mathrm{ch}x$, $x \in R$
5	$x - \frac{x^2}{2} + \frac{x^3}{3} + \ldots + (-1)^{n-1} \frac{x^n}{n} + \ldots$	$\ln(1+x)$, $x \in]-1;1]$

6	$x-\frac{x^3}{3}+\frac{x^5}{5}+\ldots+(-1)^{m-1}\frac{x^{2m-1}}{2m-1}+\ldots$	$\operatorname{arctg}x,\ x\in[-1;1]$
7	$-x-\frac{x^2}{2}-\frac{x^3}{3}-\ldots-\frac{x^n}{n}-\ldots$	$\ln(1-x),$ $x\in[-1;1[$
8	$1-x+x^2-\ldots+(-1)^n x^n+\ldots$	$\frac{1}{1+x},\ x\in]-1;1[$
9	$1+x+x^2-\ldots+x^n+\ldots$	$\frac{1}{1-x},\ x\in]-1;1[$
10	$1+\alpha x+\frac{\alpha(\alpha-1)}{2!}x^2+\ldots+\frac{\alpha(\alpha-1)\ldots(\alpha-n+1)}{n!}x^n+\ldots$	$(1+x)^\alpha, x\in]-1;1[$
11	$2\left(x+\frac{x^3}{3}+\frac{x^5}{5}+\ldots+\frac{x^{2n-1}}{2n-1}+\ldots\right)$	$\ln\frac{1+x}{1-x}, x\in]-1;1[$

3. Вычислить предел последовательности $\{a_n\}_{n=0}^{\infty}$, где $a_n=\frac{n}{\sqrt{n^2+1}+\sqrt{n^2-1}}$. За предел принять такое значение a_n, при котором $|a_{n+1}-a_n|\le\varepsilon$. Указать, на каком шаге был получен искомый предел с заданной точностью ε.

4. Вычислить с заданной точностью ε корень $n-$ой степени из положительного числа a как предел числовой последовательности $y_1, y_2, \ldots, y_k, \ldots$, построенной по формуле последовательных приближений $y_{k+1}=\frac{n-1}{n}y_k+\frac{a}{ny_k^{n-1}},\ k=0,1,\ldots;\ y_0=\frac{a+n-1}{2}$.

5. Найти наименьшее трехзначное из чисел Фибоначчи. Последовательность чисел $\{F_n\}_{n=0}^{\infty}$, где $F_0=1, F_1=1, F_n=F_{n-1}+F_{n-2}$ называется последовательностью Фибоначчи.

6. Найти на отрезке $[a,b]$ корень нелинейного уравнения вида $y=f(x)$ с точностью ε методом деления отрезка пополам.

Практические занятия

Тема 5: Действия над одномерными массивами в блок-схемах

Теоретические сведения. Наиболее широко известная структура данных – *массив*. Массив состоит из компонент одного типа, называемого *базовым*. Поэтому структура массивов *однородна*. Кроме того, массивы относят к так называемым структурам с *прямым доступом*. Для того чтобы обозначить отдельную компоненту, к имени всего массива добавляется индекс. *Индекс* – это значение специального типа, определенного как тип индекса массива. Количество индексов называется размерностью массива. Отметим, что в памяти ПЭВМ каждой компоненте массива отводится отдельное поле равных размеров, при этом, все элементы массива расположены подряд.

Если x является переменной-массивом размерности n, то отдельная компонента обозначается с помощью имени массива, за которым следует индекс требуемой компоненты – x_i. Иногда компоненты массивов называют переменными с индексами.

Обычный прием работы с массивами, в особенности с большими массивами – выборочное изменение отдельных его компонент, а не конструирование полностью нового составного значения. При этом переменная-массив рассматривается как массив составляющих переменных, и возможно присваивание значений отдельным компонентам, например, $x_i \leftarrow 0.456$.

Тот факт, что индексы массива относятся к определенному (скалярному) типу, имеет важное следствие: индексы массива можно вычислять. На место индексирующей константы можно подставлять любое индексирующее выражение; оно будет вычислено, и результат идентифицирует требуемую компоненту. Если необходимо выполнить некоторую операцию над всеми компонентами массива или над соседними компонентами некоторой части массива, то удобно воспользоваться циклом (пример 6.1). Поскольку в языке Си индексация элементов массива

производится от нуля, то при построении блок-схем будем использовать этот факт.

Пример 6.1. Составить блок-схему нахождения в одномерном массиве из n элементов порядковых номеров максимального и минимального элементов. Значение элементов и их порядковые номера вывести на экран (рис. 6.1).

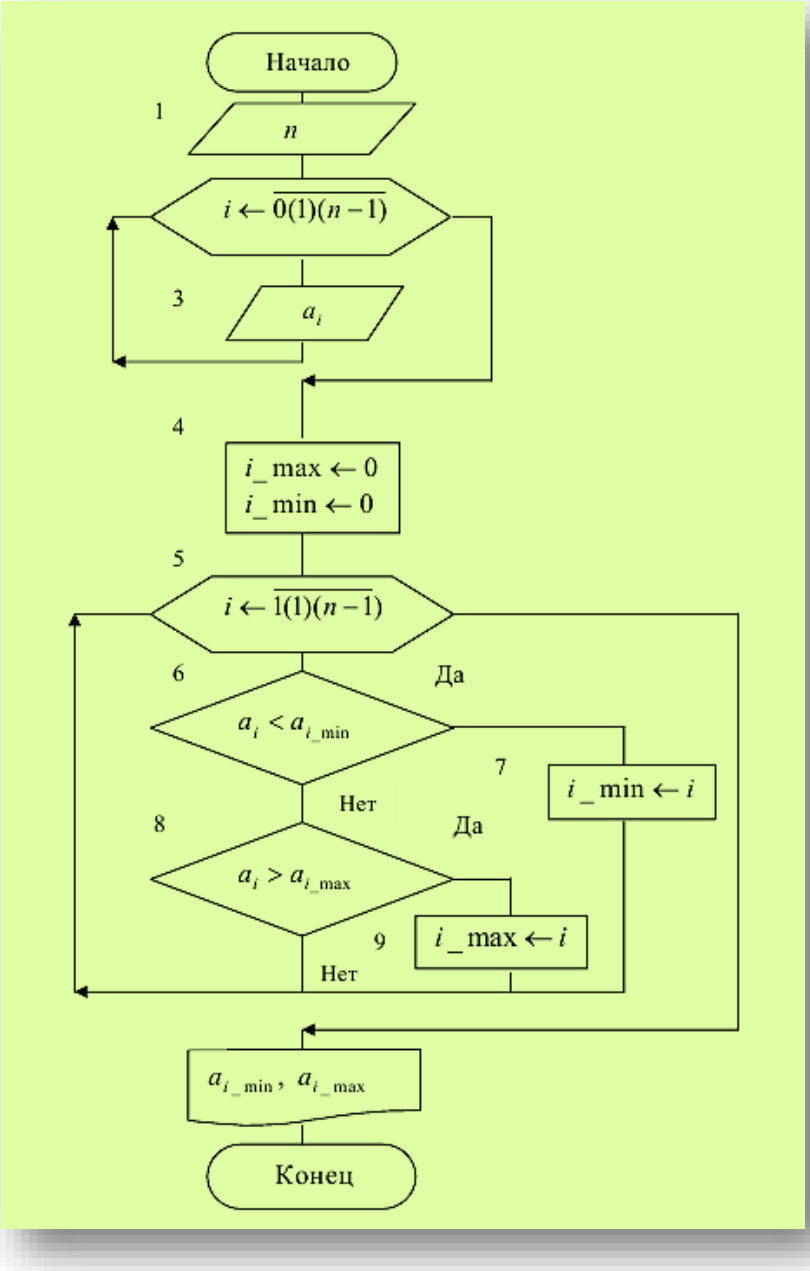


Рис. 6.1. Блок-схема примера 6.1

Пример 6.2. Составить блок-схему вычисления значения полинома $P_n(x) = a_0 x^n + a_1 x^{n-1} + a_2 x^{n-2} + \quad + a_n$ в заданной точке x . Коэффициенты, степень полинома, а также значение x ввести (рис. 6.2).

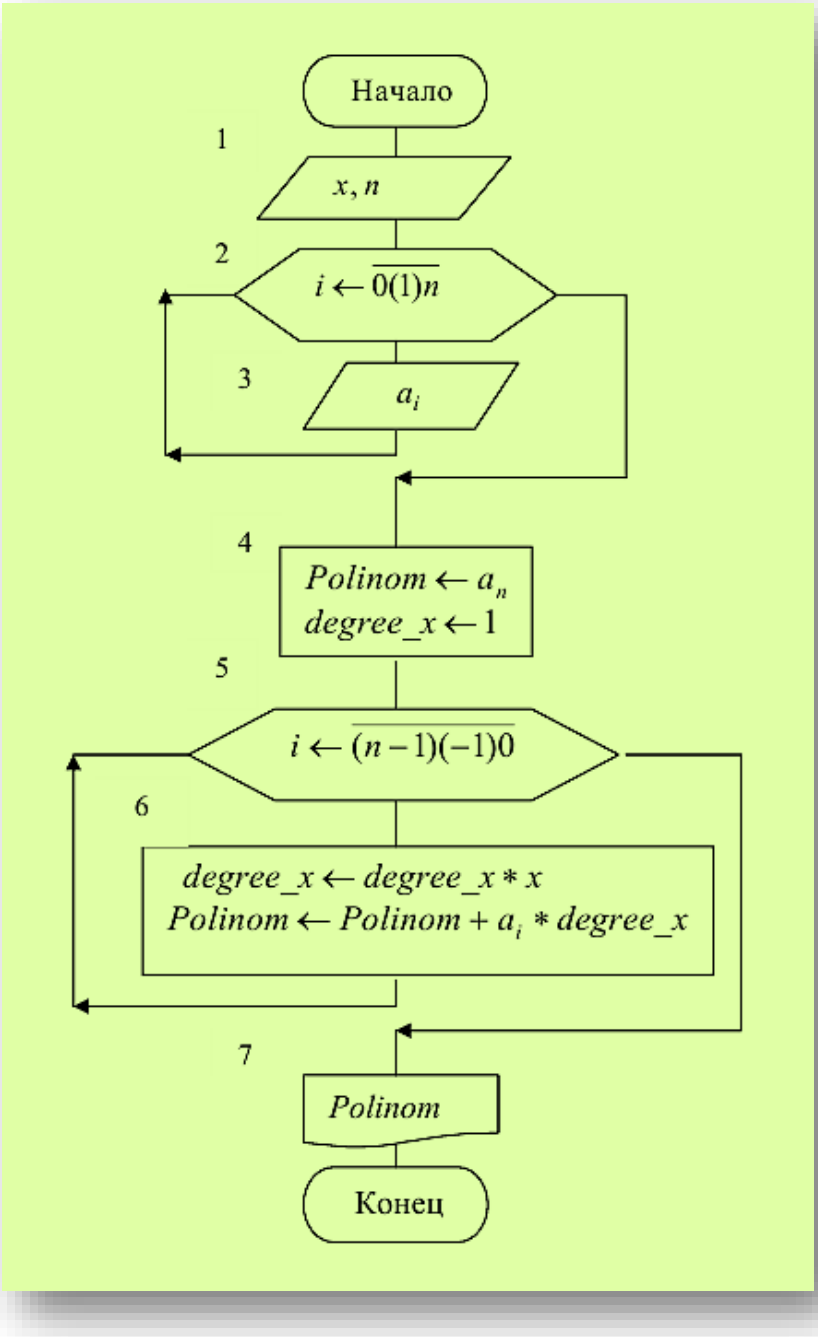


Рис. 6.2. Блок-схема примера 6.2

Сортировка массивов. В общем случае сортировку следует понимать как перегруппировку заданного множества объектов (массива, файла и т.д.) в

некотором порядке. Цель сортировки – облегчить последующий поиск элементов в таком отсортированном множестве. Выбор алгоритма сортировки зависит от структуры обрабатываемых данных. Рассмотрим основные методы сортировки применительно к одномерным массивам.

Дан одномерный массив размерности n : $a_0,a_1,...,a_{n-1}$. Сортировка данного массива есть перестановка его элементов в массив a_{k0} , $a_{k1},...,$ a_{kn-1}, где при некоторой упорядочивающей функции f выполняются отношения $f(a_{k0}) \le f(a_{k1}) \le ... f(a_{kn-1})$. Обычно упорядочивающая функция не вычисляется, а хранится как явная компонента каждого элемента. Ее значение называется ключом элемента. Основное условие при выборе метода сортировки массивов – выбранный метод должен экономно использовать доступную память. Это предполагает, что перестановки, приводящие элементы в порядок, должны выполняться на том же месте («in site»). Методы сортировки «in site» можно разбить в соответствии с определяющими их принципами на три основные типа – сортировка с помощью прямого включения; сортировка с помощью прямого выбора; сортировка с помощью прямого обмена.

Сортировка с помощью прямого включения. Метод основан на следующих принципах. Элементы условно делятся на «уже готовую» последовательность и исходную последовательность. При каждом шаге, начиная с i =1 и увеличивая i каждый раз на единицу, из исходной последовательности извлекается i − ый элемент и перекладывается в готовую последовательность, при этом, он вставляется на нужное место.

Сортировка с помощью прямого выбора. Метод основан на следующих принципах. Выбирается элемент с наименьшим значением. Он меняется с первым элементом x_0. Затем этот процесс повторяется с оставшимися n −1 элементами, n − 2 элементами и т.д. до тех пор, пока не останется один, самый больший элемент.

Сортировка с помощью прямого обмена (метод «пузырька»). Метод основан на сравнении и смене мест пары соседних элементов и продолжении этого процесса до тех пор, пока не будут упорядочены все элементы. Проходы по массиву повторяются, сдвигая каждый раз наименьший элемент оставшейся последовательности к левому концу массива. Если рассматривать

массивы как вертикальные, то элементы можно интерпретировать как пузырьки в чане с водой разного веса. В этом случае при каждом проходе один пузырек поднимается до уровня, соответствующего его весу. Такая сортировка известна под названием «пузырьковая сортировка».

Каждый из изложенных выше методов сортировки допускает оптимизацию, основанную на анализе массива (отсортирован или нет), полученного на каждом шаге алгоритма (см. лекции).

Пример 6.3. Составить блок-схему сортировки по неубыванию одномерного целочисленного массива размерности *n* методом «пузырька» (рис. 6.3).

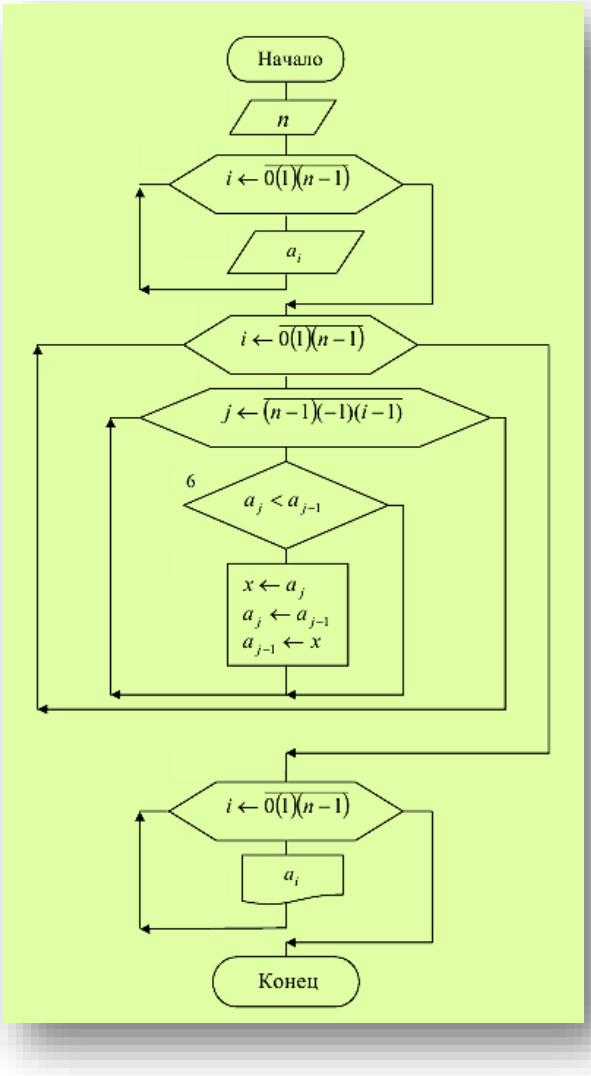


Рис. 6.3. Блок-схема примера 6.3

Практические занятия

Тема 5: Действия над одномерными массивами в блок-схемах

Варианты заданий

1. В одномерном массиве из n элементов найти порядковые номера первого отрицательного и последнего положительного элементов (если таковые имеются). Значение элементов и их порядковые номера вывести на экран или выдать соответствующее сообщение.
2. Ввести одномерный массив из n элементов. Вычислить сумму всех отрицательных чисел, их количество и сумму всех положительных чисел.
3. В зависимости от того, образуют элементы заданного массива целых чисел из n элементов строго убывающую, не возрастающую, строго возрастающую, неубывающую последовательность, выдать соответствующее сообщение.
4. Ввести одномерный массив из n элементов. Сформировать на его месте новый массив, в котором первым элементом будет последний элемент старого, вторым – предпоследний и т.д.
5. Элемент называется локальным минимумом (максимумом), если у него нет соседа, меньшего (большего), чем он сам. Найти все локальные минимумы и максимумы в заданном массиве из n элементов.
6. В неубывающей последовательности из n элементов найти количество элементов, меньших заданного числа и напечатать их.
7. Элементы одномерного массива из n элементов циклически сдвинуть на k мест влево (вправо).
8. Выполнить попарное суммирование произвольного конечного ряда чисел следующим образом. На первом этапе суммировать попарно рядом стоящие числа, на втором - результаты первого этапа аналогичным образом, и т.д., пока не останется одно число.
9. Ввести одномерный массив $a_0, a_1,..., a_{n-1}$. Вычислить все суммы вида $S_i = a_i + a_{i+1} + ...+ a_j, \quad j, \quad 0 \le i \le j \le n-1$ и среди них определить максимальную сумму.
10. Даны два вектора размерности n. Вычислить их скалярное произведение.

11. Дан вектор размерности n. Пронормировать его по максимальному элементу.
12. Сложить два полинома заданных степеней (коэффициенты хранятся в массивах).
13. Сложить два полинома заданных степеней (коэффициенты хранятся в массивах).
14. В массиве из n элементов выбрать без повторений все элементы, встречающиеся более одного раза.
15. Ввести одномерный массив из n элементов. Определить число различных элементов в нем.
16. Вычислить коэффициенты от $1-$ ой до $m-$ ой ($m \le n$) производной для полинома степени n $P_n(x) = a_0 x^n + a_1 x^{n-1} + a_2 x^{n-2} + \ + a_n$.
17. Ввести одномерный массив из n элементов. Отсортировать массив по неубыванию (невозрастанию) методом прямого включения.
18. Ввести одномерный массив из n элементов. Отсортировать массив по неубыванию (невозрастанию) методом прямого выбора.
19. Ввести одномерный массив из n элементов. Отсортировать массив по неубыванию (невозрастанию) методом прямого обмена.
20. Ввести одномерный массив из n элементов. Отсортировать массив по неубыванию (невозрастанию) методом шейкер-сортировки.
21. Ввести одномерный массив из n элементов. Отсортировать массив по неубыванию (невозрастанию) методом быстрой сортировки.
22. Ввести одномерный массив из n элементов. Отсортировать массив по неубыванию (невозрастанию) методом Шелла.
23. Ввести одномерный массив из n элементов. Отсортировать массив по неубыванию (невозрастанию) методом пирамидальной сортировки.

При защите заданий по данной теме необходимо:

- **изложить основные принципы методов сортировки и поиска, использовавшихся при построении блок-схемы;**
- **уметь пояснить каждый из переходов в блок-схеме в соответствии с выбранным методом решения.**

По данной теме построение блок-схемы можно заменить предоставлением программы, работоспособность которой будет проверена (и это приветствуется).

Практические занятия

Тема 6: Действия над матрицами в блок-схемах

Теоретические сведения. Матрицы представляются двумерным массивом или одномерным массивом, элементами которого являются одномерные массивы. Матрицу удобно трактовать как таблицу из n строк и m столбцов, записанную в виде:

$$A = \begin{bmatrix} a_{00} & a_{01} & \dots & a_{0,m-1} \\ a_{10} & a_{11} & \dots & a_{1,m-1} \\ \dots & \dots & \dots & \dots \\ a_{n-1,0} & a_{n-1,1} & \dots & a_{n-1,m-1} \end{bmatrix}$$

Элементы матрицы A размерности $n \times m$ можно записать в виде a_{ij}, $i = \overline{0,n-1}$, $j = \overline{0,m-1}$.. Здесь первый индекс i указывает положение элемента матрицы в строке, а второй индекс j - в столбце. Очевидно, что для просмотра всех элементов матрицы необходимо использование вложенных циклов.

Пример 7.1. Дана матрица $A(n \times m)$. Составить блок-схему нахождения первого по порядку максимального элемента матрицы (рис. 7.1.).

Пример 7.2. Дана квадратная матрица $A(n \times n)$. Транспонировать матрицу без использования вспомогательного массива (на том же месте) (рис. 7.2).

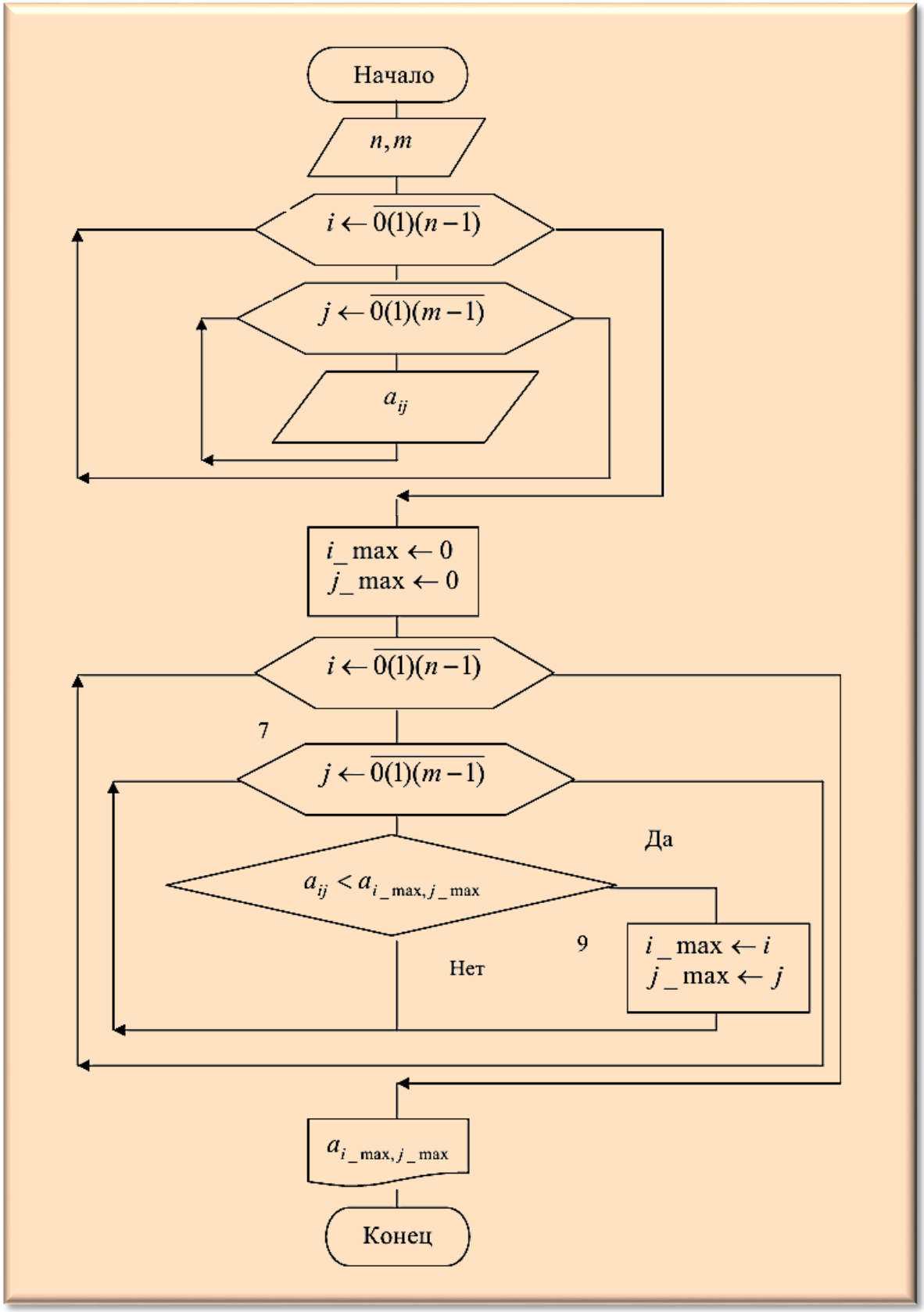


Рис. 7.1. Блок-схема примера 7.1

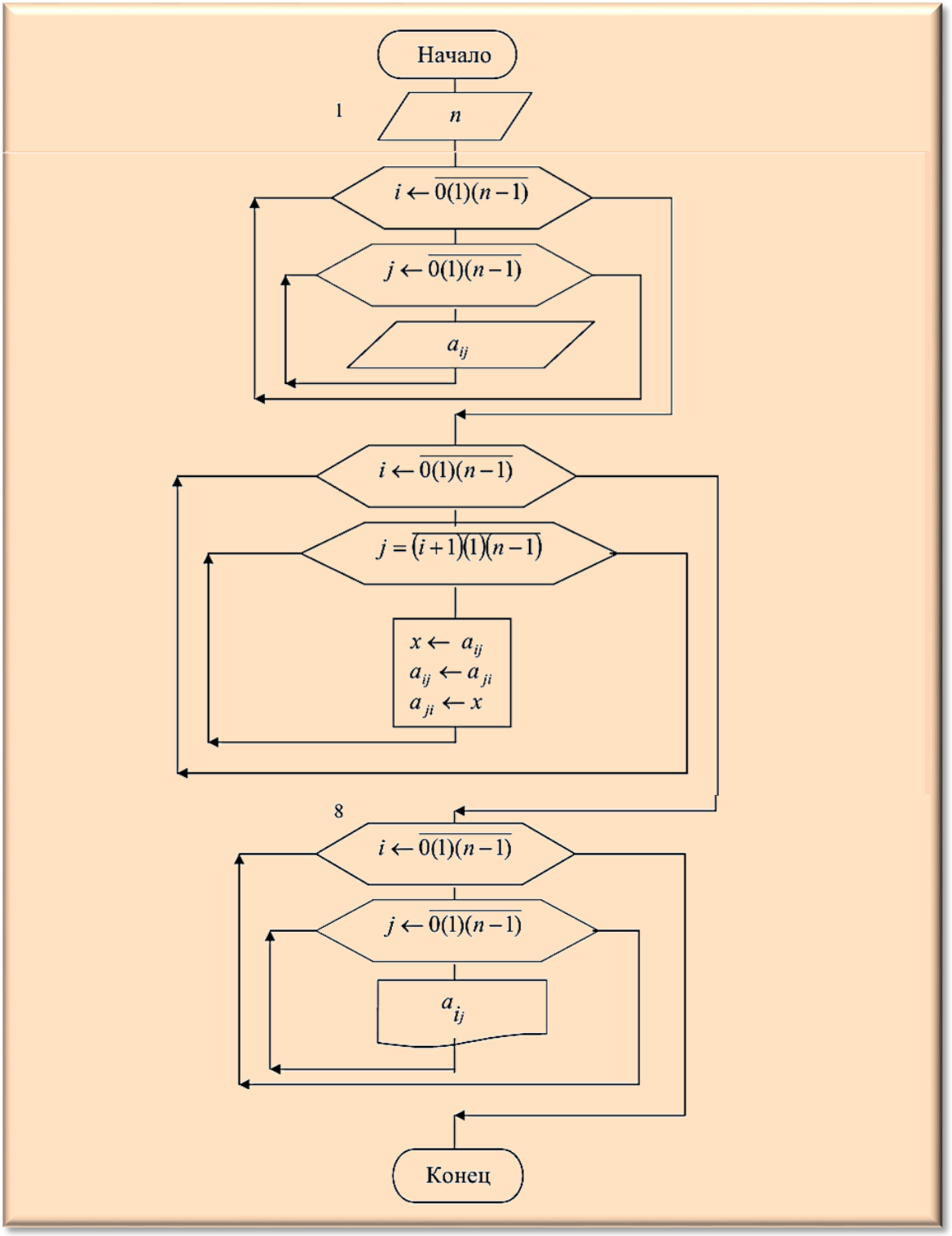


Рис. 7.2. Блок-схема примера 7.2

Практические занятия

Тема 6: Действия над матрицами в блок-схемах

Варианты заданий

1. Дана матрица $A(n \times m)$. Найти порядковые номера первого отрицательного и последнего положительного элемента (если таковые имеются). Значение элементов и их порядковые номера вывести на экран или выдать соответствующее сообщение.
2. Дана матрица $A(n \times m)$. Найти количество отрицательных, положительных и нулевых элементов в ней.
3. Дано натуральное число n . Построить матрицу $A(n \times n)$ вида:

а)

$$\begin{bmatrix} n & 0 & \dots & 0 \\ n-1 & n & \dots & 0 \\ \dots & \dots & \dots & \dots \\ 1 & 2 & \dots & n \end{bmatrix}$$

б)

$$\begin{bmatrix} 1 & 1 & 1 & \dots & 1 & 1 & 1 \\ 0 & 1 & 1 & \dots & 1 & 1 & 1 \\ 0 & 0 & 1 & \dots & 1 & 0 & 0 \\ \dots & \dots & \dots & \dots & \dots & \dots & \dots \\ 0 & 0 & 1 & \dots & 1 & 0 & 0 \\ 0 & 1 & 1 & \dots & 1 & 1 & 0 \\ 1 & 1 & 1 & \dots & 1 & 1 & 1 \end{bmatrix}$$

в)

$$\begin{bmatrix} 1 & 0 & 0 & \dots & 0 & 0 & 1 \\ 1 & 1 & 0 & \dots & 0 & 1 & 1 \\ 1 & 1 & 1 & \dots & 1 & 1 & 1 \\ \dots & \dots & \dots & \dots & \dots & \dots & \dots \\ 1 & 1 & 1 & \dots & 1 & 1 & 1 \\ 1 & 1 & 0 & \dots & 0 & 1 & 1 \\ 1 & 0 & 0 & \dots & 0 & 0 & 1 \end{bmatrix}$$

г)

$$\begin{bmatrix} 1 & 0 & 0 & \dots & 0 & 0 & n \\ 0 & 2 & 0 & \dots & 0 & n-1 & 0 \\ 0 & 0 & 3 & \dots & n-2 & 0 & 0 \\ \dots & \dots & \dots & \dots & \dots & \dots & \dots \\ 0 & 0 & 3 & \dots & n-2 & 0 & 0 \\ 0 & 2 & 0 & \dots & 0 & n-1 & 0 \\ 1 & 0 & 0 & \dots & 0 & 0 & n \end{bmatrix}$$

4. Дан массив A(n). Построить матрицу $A(n \times n)$ вида

$$\begin{bmatrix} a_1 & a_2 & \dots & a_{n-1} & a_n \\ a_2 & a_3 & \dots & a_n & a_1 \\ \dots & \dots & \dots & \dots & \dots \\ a_n & a_1 & \dots & a_{n-2} & a_{n-1} \end{bmatrix}$$

5. По массиву X(n) построить матрицу $A(n \times n)$, где $a_{ij} = x_i \cdot x_j$, $i, j = 1, n$. Вычислить $\sqrt{\sum_{i=1}^{n}\sum_{j=1}^{n} a_{ij}^2}$.
6. По массиву X(n) построить матрицу $A(n \times n)$, где $a_{ij} = x_i \cdot x_j$, $i, j = 1, n$. Вычислить $\max\limits_{1 \le i \le n} \sum_{j=1}^{n} |a_{ij}|$.
7. Рассматривая матрицу $A(n \times n)$ как составленную из четырех квадратов, переставить в ней друг с другом центрально-симметричные квадраты.
8. Рассматривая матрицу $A(n \times n)$ как составленную из четырех квадратов, переставить в ней друг с другом квадраты по кругу.
9. Дана матрица $A(n \times m)$. Найти матрицу, полученную перестановкой столбцов (первого с последним, второго с предпоследним и т.д.) из данной.
10. Дана матрица $A(n \times m)$. Найти матрицу, полученную перестановкой строк (первой с последней, второй с предпоследней и т.д.) из данной.
11. Найти все седловые точки матрицы $A(n \times m)$. Матрица имеет седловую точку a_{ij}, если a_{ij} является минимальным элементом в i-ой строке и максимальным элементом в j-ом столбце.
12. В матрице $A(n \times m)$ найти максимальный элемент и путем перестановки строк и столбцов поместить его на место элемента a_{00}.
13. Среди строк целочисленной матрицы $A(n \times m)$, содержащих только четные элементы, найти строку с максимальной суммой модулей элементов.

14. Среди столбцов целочисленной матрицы $A(n \times m)$, содержащих только положительные элементы, найти столбец с минимальным произведением элементов.
15. Известно, что в каждой строке и каждом столбце квадратной матрицы имеется единственный отрицательный элемент. Переставить строки матрицы так, чтобы в полученной матрице отрицательные элементы находились на главной диагонали.
16. Известно, что в матрице $A(n \times m)$ нет нулей. Заменить повторяющиеся в матрице элементы нулями.
17. Дана матрица $A(n \times n)$, элементы которой различны. Найти наибольший элемент среди стоящих на главной и побочной диагоналях и поменять его местами с элементом, стоящим на пересечении этих диагоналей.
18. Найти максимальный (минимальный) элемент среди тех строк матрицы $A(n \times m)$, элементы которых упорядочены по неубыванию (невозрастанию).
19. Найти минимальный (максимальный) элемент среди тех столбцов матрицы $A(n \times m)$, элементы которых упорядочены по невозрастанию (неубыванию).
20. Расположить строки матрицы $A(n \times m)$ в порядке убывания модулей сумм элементов строк.
21. Из матрицы $A(n \times m)$ получить на том же месте матрицу, вычеркнув из исходной матрицы строку и столбец, на пересечении которых находится первый по порядку максимальный элемент.
22. Заполнить матрицу $A(n \times n)$ по спирали массивом натуральных чисел 1,2 ,..., $n \times n$, начиная с элемента, находящегося в верхнем левом углу.
23. Начиная с центра, обойти по спирали все элементы квадратной матрицы $A(n \times n)$, распечатывая их в порядке обхода.
24. Характеристикой строки целочисленной матрицы $A(n \times m)$ назовем сумму её положительных четных элементов. Переставляя строки заданной матрицы, расположить их в соответствии с ростом их характеристик.

25. Характеристикой столбца целочисленной матрицы $A(n \times m)$ назовем сумму модулей его отрицательных нечетных элементов. Переставляя столбцы заданной матрицы, расположить их в соответствии с ростом их характеристик.
26. В матрице $A(n \times n)$ элементы каждой строки верхнего треугольника упорядочить по возрастанию, нижнего - по убыванию, диагональные - оставить без изменений.
27. Для заданной матрицы $A(n \times n)$ найти максимум среди сумм элементов диагоналей, параллельных главной диагонали, и минимум среди сумм модулей элементов диагоналей, параллельных побочной диагонали.

Способ выполнения заданий (либо блок-схема, либо программа) – по выбору студента

Практические занятия

Тема 7: Представление информации в ПЭВМ типа IBM PC/AT

Теоретические сведения. Информация в памяти ЭВМ хранится и обрабатывается в двоичном виде. Форма записи данных в памяти машины называется *внутренним представлением информации* в ЭВМ. Единицей хранения информации является один бит, т.е. двоичный разряд, который может принимать значения 0 или 1. Применение двоичной системы счисления позволяет использовать для хранения информации элементы, имеющие всего два устойчивых состояния. Одно состояние служит для изображения единицы соответствующего разряда числа, другое – для изображения нуля. Обычно биты памяти группируются в более широкие структуры. Группа из восьми битов называется байтом. Поля памяти ЭВМ имеют специальные названия: 16 бит – слово, 32 бита – двойное слово, 1024 байта (1 К байт) – лист. Все байты пронумерованы, начиная с нуля.

Адресом любой информации считается адрес (номер) самого первого байта поля памяти, выделенного для ее хранения.

Существует два основных способа представления чисел, называемых представлением с *фиксированной* и с *плавающей* точкой.

Двоичные числа с фиксированной точкой. Для чисел с фиксированной точкой положение точки зафиксировано после младшей цифры числа, дробная часть отсутствует, точка в изображении числа опускается. Таким образом, в виде с фиксированной точкой могут храниться только целые числа (в памяти ЭВМ они записываются в двоичном виде). Следует отметить, что в алгоритмических языках для записи числа в память используются специальные слова – *атрибуты*. В языке Си, например, для указания того, что значение переменной должно быть записано с фиксированной точкой, применяют атрибуты **int**, **short int**, **long int**, **unsigned int**.

Целое двоичное число занимает в памяти 16 или 32 двоичных разряда. Это зависит от длины числа и способа представления переменной.

Рассмотрим, как записывается число в двойном слове (в слове оно записывается аналогично). Седьмой бит первого байта является служебным при хранении чисел со знаком, и его содержимое не оказывает влияния на абсолютную величину числа. Отметим, что для положительных чисел содержимым служебного бита является нуль. Младший двоичный разряд числа записывается в 0-ой бит, т.е. число заполняют справа налево. Если число положительно, то оставшиеся биты заполняются нулями.

Например, число $127_{(10)}$ в 4 байтах будет представлено следующим образом:

$$127_{(10)} = 7F_{(16)} = 1111111_{(2)} = 1 \cdot 2^6 + 1 \cdot 2^5 + \ldots + 1 \cdot 2^1 + 1 \cdot 2^0$$

7 6 5 4 3 2 1 0	7 6 5 4 3 2 1 0	7 6 5 4 3 2 1 0	7 6 5 4 3 2 1 0	биты
0000 0000	**0000 0000**	**0000 0000**	**01111111**	байты
1 – ый байт	2–ой байт	3–ий байт	4–ый байт	

Из приведенного выше представления видно, что в младшем разряде записан коэффициент при 2^0, в следующем – при 2^1 и т.д. Очевидно, что если во всех битах с 0-го по 30-ый находятся 1, то максимальное целое положительное число, которое можно записать в двойном слове, имеет вид:

$$01111111\ \ 11111111\ \ 11111111\ \ 11111111 = 2^{31} - 1 = 2147483647_{(10)}.$$

Форму записи положительных целых двоичных чисел называют *прямым кодом*. В этом случае их запись в отведенной памяти полностью соответствует символической записи в двоичной системе счисления.

Отрицательные числа записываются в *дополнительном коде*. Использование этого кода позволяет упростить аппаратную реализацию операции вычитания, которая заменяется операцией сложения уменьшаемого, представленного в прямом коде, и вычитаемого, представленного в дополнительном коде. *Дополнительный код* получается из прямого путем инвертирования каждого бита (*обратный код*) и затем добавления 1 к младшему биту числа.

Представим число $-95_{(10)}$ как двоичное число с фиксированной точкой.

$$-95_{(10)} = -5F_{(16)} = -1011111_{(2)}$$

1. Запишем прямой код числа в четыре байта:

   00000000 00000000 00000000 01011111

2. Запишем обратный код числа:

   11111111 11111111 11111111 10100000

3. Прибавив к младшему биту 1, получим дополнительный код числа:

$$\begin{array}{llll} 11111111 & 11111111 & 11111111 & 10100000 \\ + & & & \\ & & & 1 \\ \hline \underbrace{\mathbf{11111111}}_{\text{1-ый байт}} & \underbrace{\mathbf{11111111}}_{\text{2-ой байт}} & \underbrace{\mathbf{11111111}}_{\text{3-ий байт}} & \underbrace{\mathbf{10100001}}_{\text{4-ый байт}} \end{array}$$

При хранении целых положительных чисел существует возможность почти вдвое увеличить числовой диапазон. Это достигается за счет использования служебного седьмого бита первого байта для хранения старшего коэффициента в двоичном представлении числа. Так, например, число $65535_{(10)} = FFFF_{(16)} = 11111111\ \ 11111111_{(2)}$, объявленное как целое без знака, в двух байтах запишется в виде

$$\underbrace{11111111}_{1-\text{ый байт}}\ \underbrace{11111111}_{2-\text{ой байт}}$$

Двоичные числа с плавающей точкой. Числа, в которых положение точки не зафиксировано после некоторого разряда, а указывается специальным числом, называются числами с плавающей точкой.

В общем виде любое число A в системе счисления с основанием p можно представить в виде $A = m \cdot p^k$, где m - мантисса числа A, k - порядок числа. При этом если мантисса числа удовлетворяет неравенству $1 \le |m| < p$, то число *нормализованное*.

Размеры областей, выделяемых под числа с плавающей точкой, существенно зависят от аппаратной реализации вычислительной техники. Для IBM PC/AT, например, числа с плавающей точкой могут, в зависимости от объявленных атрибутов **float**, **double**, **long double**, располагаться соответственно в 32, 64 и 80 битах памяти в виде:

float	±	характеристика		нормализованная мантисса	
биты	31	30	23	22	0

double	±	характеристика		нормализованная мантисса	
биты	63	62	52	51	0

long double	±	характеристика		нормализованная мантисса	
биты	79	78	64	63	0

Мантисса числа записывается в нормализованном виде (в двоичном представлении числа перед точкой сохраняется один значащий двоичный бит). Чтобы увеличить диапазон представления чисел в форматах float и double, единица перед точкой в двоичной нормализованной записи числа в память не записывается; в формате long double – единица сохраняется. Знак мантиссы записывается в последнем бите первого байта.

С целью упрощения аппаратной реализации арифметических операций в представлении числа с плавающей точкой знак порядка явно не хранится. Вместо порядка в память записывается величина, называемая *характеристикой*. Характеристика получается из порядка путем прибавления поправочного коэффициента, который для чисел типа float равен $127_{(10)} = 7F_{(16)}$, для double – $1023_{(10)} = 3FF_{(16)}$, для long double – $16383_{(10)} = 3FFF_{(16)}$.

Представим число $-18.2_{(10)} = -10010.(0011)_{(2)}$, нормализованная запись которого $-1.0010(0011)_{(2)} \cdot 2^{100}$, где нормализованная мантисса $m = -1.0010(0011)_{(2)}$, а порядок $k = 4_{(10)} = 100_{(2)}$. Перейдем от порядка к его характеристике: $4 + 127 = 131_{(10)} = 83_{(16)} = 1000\,0011_{(2)}$. Получим представление числа $18.2_{(10)}$ в формате float:

1	**10000011**	**00100011001100110011010**
31	30 23	22 0

Если количество цифр в числе, записываемом в ПЭВМ или полученном в результате вычислений, больше выделенного поля памяти, то избыточные цифры отбрасываются, а число, записываемое в память, округляется с избытком. В формате double характеристика равна $4 + 1023 = 1027_{(10)} = 403_{(16)} = 100\,0000\,0011_{(2)}$, и число представимо в виде:

1	**10000000011**	**0010001100110011…00110011**
63	62 52	51 0

Размеры выделенных для хранения данных областей определяют интервал допустимых для этого атрибута значений – диапазон (Таблица 2.1). Всякое число, меньшее по абсолютной величине положительного минимального числа, представленного в соответствующем формате, будет записано в память в виде нуля. Для данного формата это так называемый машинный нуль. Кроме того, числа данного формата не должны превышать по абсолютной величине максимальное число, представленное в этом формате. В противном случае старшие биты числа будут потеряны, а результат не верен. Описанная ситуация называется переполнением разрядной сетки, а сами числа – машинной бесконечностью.

Таблица 2.1.

Тип данных	Кол-во бит	Диапазон	
		Abs(min)	Abs(max)
float	32	$3.4 \cdot 10^{-38}$	$3.4 \cdot 10^{38}$
double	64	$1.7 \cdot 10^{-308}$	$1.7 \cdot 10^{308}$
long double	80	$3.4 \cdot 10^{-4932}$	$3.4 \cdot 10^{4932}$

Символьная информация. В ПЭВМ для внутреннего представления символьных данных используется так называемый ASCII код, согласно которому каждому символу соответствует 8-разрядный код, т.е. в один байт записывается один символ. Например, текст «1486DX2» в памяти имеет вид:

00110001	00110100	00111000	00110110	01000100	01011000	00110010
1	4	8	6	D	X	2

Таблицы ASCII, история развития

Помимо цифр, на мониторы ЭВМ необходимо выводить еще и множество символов. Ясно, что для вывода каждого символа необходим некий машинный код, однозначно соответствующий этому символу, или некое правило, по которому можно организовать корректный вывод каждого символа на дисплей. Разумеется, разрабатывать такую систему ввода-вывода следует оптимальным образом с точки зрения потребления ресурсов компьютера. Особенно важно в этом случае помнить о том, что производительность компьютеров в отдаленные времена зарождения вычислительной техники была ничтожной, с современных позиций, а системные программисты и разработчики аппаратной части боролись за каждый бит, адрес, инструкцию, регистр, освобождая оперативную память и адресное пространство компьютерных "малышей".

Давайте подсчитаем, сколько необходимо символов для вывода информации на дисплей. Исторически сложилось так, что первые разработчики компьютеров были носителями английского языка. Что им

было необходимо обеспечить для вывода на монитор? Во-первых, 26 букв английского алфавита (строчных), во-вторых, 26 прописных, 9 знаков препинания (. , : ! " ; ? ( )), пробел, 10 цифр, 5 знаков арифметических действий (+,-,*, /, ^) и специальные символы (№ % _ # $, и так далее ^, &, >, <, |, \). Получается чуть больше сотни символов. Такой сравнительно небольшой базовый набор символов можно закодировать при помощи таблиц соответствия этого набора машинным кодам (фактически, двоичным числам). Можно вполне ограничиться набором двоичных чисел от **0** до $\mathbf{2^7}$ (всего 128 позиций), что и было сделано. Таблица соответствия получила название **ASCII** (**A**merican **S**tandard **C**ode for **I**nformation **I**nterchange). В рамках таблицы **ASCII** создание многоязычных документов являлось очень проблематичной, а в большинстве случаев и совершенно невыполнимой задачей.

ASCII (англ. *American Standard Code for Information Interchange*) — американская стандартная кодировочная таблица для печатных символов и некоторых специальных кодов. В американском варианте нашего английского языка произносится [*э́ски*], тогда как в Великобритании чаще произносится [*а́ски*]; по-русски произносится также [*а́ски*] или [*аски́*].

Рассмотрим структуру таблицы ASCII. Для удобства символы в ней пронумерованы в шестнадцатеричной системе счисления от 0 - 7F. Первые знакоместа в таблице занимают непечатаемые символы (0 до 1F), затем следуют печатаемые символы (20 - 7F).

Управляющие (непечатаемые) символы - элементы данной кодировки, которым не приписано графическое представление, но которые используются для управления устройствами, организации передачи данных и других целей.

Сейчас для этих целей применяются форматы файлов, языки управления устройствами (такие как Postscript) и сетевые протоколы. Поэтому многие управляющие символы сейчас или не используются вообще, или используются не по назначению.

Номер	Английское название	Русское название	Сочетание клавиш	Назначение
00	NULL	пустой символ		Этот символ ничего не делает. Некоторые терминалы изображают его как пробел, но это неправильно. Часто NULL используют для обозначения конца цепочки символов (например, в языке C).
01	START OF HEADING	начало заголовка	^A	В настоящее время используется в консоли маршрутизаторов Cisco.
02	START OF TEXT	начало текста	^B	В настоящее время используется в консоли маршрутизаторов Cisco.
03	END OF TEXT	конец текста	^C	При вводе на терминале обычно интерпретируется как сигнал прерывания.
04	END OF TRANSMISSION	конец передачи	^D	При вводе на терминале в UNIX-системах интерпретируется как конец вводимых данных. Если текущая программа брала данные с терминала, то она завершается, как только обработает всё, что было до символа ^D.
05	ENQUIRY	запрос	^E	Использовался в телетайпной связи. В ответ предполагалось получить идентификационную строку удалённого аппарата.
06	ACKNOWLEDGE	подтверждение	^F	Использовался в телетайпной связи.
07	BELL	звуковой сигнал	^G	Если этот символ послать на принтер или на терминал, то ничего не напечатается, но послышится звуковой сигнал.
08	BACKSPACE	возврат на шаг	^H	Перемещает позицию печати на один символ назад. На принтерах может использоваться для наложения одного символа на другой, например **a** BS ^ = **â**. При вводе с терминала иногда используется для стирания предшествующего символа («забой»).

09	CHARACTER TABULATION (horizontal tabulation)	горизонтальная табуляция	^I	Перемещает позицию печати к следующей позиции горизонтальной табуляции.
0A	LINE FEED	перевод строки	^J	Перемещает позицию печати на одну строку вниз. В текстовых файлах в Unix разделяет строки.
0B	LINE TABULATION (vertical tabulation)	вертикальная табуляция	^K	Перемещает позицию печати к следующей позиции вертикальной табуляции. На терминалах этот символ обычно эквивалентен переводу строки.
0C	FORM FEED	прогон страницы, смена страницы	^L	Выбрасывает текущую страницу и начинает печать со следующей. На терминалах этот символ обычно эквивалентен переводу строки (хотя в принципе можно было бы его использовать для очистки экрана).
0D	CARRIAGE RETURN	Возврат каретки	^M	Перемещает позицию печати в крайнее левое положение. В текстовых файлах в Mac OS (но не Mac OS X) разделяет строки. В CP/M, MS-DOS и Microsoft Windows для разделения строк в текстовых файлах используется сочетание символов \r\n.
0E	SHIFT OUT (locking-shift one)	режим национальных символов	^N	В КОИ-7 включает русский режим. На некоторых принтерах включает режим символов двойной ширины.
0F	SHIFT IN (locking-shift zero)	режим обычного ASCII	^O	В КОИ-7 включает латинский режим. На некоторых принтерах включает режим узких символов.
10	DATA LINK ESCAPE	освобождение канала данных	^P	Означает, что следующий за ним управляющий символ должен восприниматься как данные, а не как управляющий символ.
11	DEVICE CONTROL ONE	1-й код управления устройством	^Q	На терминалах разрешает продолжить вывод данных.
12	DEVICE CONTROL TWO	2-й код управления устройством	^R	

13	DEVICE CONTROL THREE	3-й код управления устройством	^S	На терминалах временно прерывает (приостанавливает) вывод данных.
14	DEVICE CONTROL FOUR	4-й код управления устройством	^T	
15	NEGATIVE ACKNOWLEDGE	отрицательное подтверждение	^U	Использовался в телетайпной связи.
16	SYNCHRONOUS IDLE	пустой символ для синхронного режима передачи	^V	Некоторые линии связи устроены так, что требуют непрерывной передачи данных. Если передавать нечего, то передают этот символ.
17	END TRANSMISSION BLOCK	конец блока передаваемых данных	^W	
18	CANCEL	отмена	^X	Данные, которые идут перед ним, некорректны. (Обычно речь идёт об одной строке.)
19	END OF MEDIUM	конец носителя	^Y	Использовался, напр., если закончилась перфолента и т.п.
1A	SUBSTITUTE	символ замены	^Z	Ставится на месте символов, значения которых были потеряны при передаче. В CP/M и MS-DOS использовался для обозначения конца текстовых файлов и конца вводимых с консоли данных (хотя для этого были предназначены символы ^C и ^D). Некоторые текстовые редакторы под DOS автоматически добавляли в конце файла ^Z.
1B	ESCAPE	Альтернативный регистр №2 (AP2)	^[	Означает, что следующие за ним символы имеют какое-то другое значение, отличное от того, которое определено в ASCII. Обычно начинает управляющие последовательности.

1C	INFORMATION SEPARATOR FOUR (file separator)	разделитель данных № 4 (разделитель файлов)	^\	
1D	INFORMATION SEPARATOR THREE (group separator)	разделитель данных № 3 (разделитель групп)	^]	
1E	INFORMATION SEPARATOR TWO (record separator)	разделитель данных № 2 (разделитель записей)	^^	Видимо, предназначался для разделения записей в базах данных, но практически никогда не используется для этого.
1F	INFORMATION SEPARATOR ONE (unit separator)	разделитель данных № 1 (разделитель полей)	^_	Видимо, предназначался для разделения полей в базах данных, но практически никогда не используется для этого.
7F	DELETE	удаление	^?	Предназначен для забивания ошибочно пробитых символов на семидорожечных перфолентах (поскольку обозначается пробитием дырочек во всех дорожках), поэтому там он эквивалентен пустому символу (\0). На терминалах может генерироваться нажатием либо кнопки Backspace, либо кнопки Delete.

Печатные символы - элементы набора символов, имеющие графическое представление, например, в виде значка на бумаге или определённого рисунка на экране. Примерами печатных символов являются буквы, цифры, знаки препинания и псевдографические символы.

Вообще говоря, печатный символ это такой символ, при получении которого головка АЦПУ должна изобразить нечто на бумаге, после чего передвинуться на одну позицию вправо. Печатные символы противопоставляются управляющим символам, на которые устройства вывода текста должны реагировать иначе.

Число	Символ	Число	Символ	Число	Символ	Число	Символ	Число	Символ	Число	Символ
20	пробел	30	.	40	@	50	P	60	'	70	p
21	!	31	0	41	A	51	Q	61	a	71	q
22	"	32	1	42	B	52	R	62	b	72	r
23	#	33	2	43	C	53	S	63	c	73	s
24	$	34	3	44	D	54	T	64	d	74	t
25	%	35	4	45	E	55	U	65	e	75	u
26	&	36	5	46	F	56	V	66	f	76	v
27	'	37	6	47	G	57	W	67	g	77	w
28	(	38	7	48	H	58	X	68	h	78	x
29	)	39	8	49	I	59	Y	69	i	79	y
2A	*	3A	9	4A	J	5A	Z	6A	j	7A	z
2B	+	3B	:	4B	K	5B	[	6B	k	7B	{
2C	,	3C	;	4C	L	5C	\	6C	l	7C	|
2D	-	3D	<	4D	M	5D	]	6D	m	7D	}
2E	.	3E	>	4E	N	5E	^	6E	n	7E	~
2F	/	3F	?	4F	O	5F	_	6F	o	7F	

Однако базового набора кодов стало быстро не хватать. Возросший дефицит знакомест в стандартной таблице **ASCII** потребовал ее немедленного расширения. В результате возникла новая таблица кодировок, получившая название "**расширенная таблица ASCII**", число знакомест в которой возросло до $\mathbf{2^8}$ (256 знакомест). Эта таблица получила название международного стандарта IS 646, а восьмибитный код - **Latin-1**. В него были добавлены в основном латинские буквы со штрихами и диакритические символы. Вскоре появился новый стандарт IS 8859, в котором вводилось понятие "кодовая страница", т.е. набор из 256 символов для определения языка или группы языков, т.е. IS 8859-1 это **Latin-1**, IS 8859-2 включал славянские языки с латинским алфавитом (чешский, польский, вергерский), IS 8859-3 включал турецкий, мальтийский, эсперанто, галисийский языки, и т.д. Недостатком такого подхода является то, что программное обеспечение должно следить за кодовыми страницами, смешивать языки при этом невозможно, кроме того не были созданы кодовые страницы японского и китайского языков.

UNICODE

В январе 1991 года возник консорциум UNICODE (Unicode Consortium), целью которого является продвижение, развитие и реализация стандарта Unicode как международной системы кодирования для обмена информацией, а также поддержание качества этого стандарта в будущих версиях.

Стандарт UNICODE 4.0 представляет собой новую систему кодирования символов, выводимых на экран монитора или на принтер, позволяющую закодировать 1 114 112 символов (в стандарте их принято называть code points). Большинство символов, используемых в основных языках мира, занимают 65 536 code points, образуя Basic Multilingual Plane (BMP) (Основная многоязычная матрица). Оставшиеся (более миллиона) code points вполне достаточно для кодирования всех известных символов, включая малораспространенные языки и исторические знаки. Стандарт UNICODE поддерживается тремя формами, 32-битной (UTF-32), 16-битной (UTF-16) и 8-битной (UTF-8). Восьмибитная форма UTF-8 была разработана для удобной совместимости с ASCII-ориентированными системами кодирования. Стандарт UNICODE совместим с Международным стандартом International Standard ISO/IEC 10646.

Наиболее просто устроена форма UTF-32. В ней каждый символ закодирован при помощи 32-битного блока. Благодаря этому каждый символ UTF-32 обладает однозначным соответствием между декодированным символом и блоком кода. Эта форма имеет фиксированную длину знакоместа. Она покрывает все кодовое пространство UNICODE - **$0...10FFFF_{16}$.** Это гарантирует полную совместимость с UTF-16 и UTF-8. Форма UTF-32 является наиболее предпочитаемой для большинства UNIX платформ.

Стандарт UNICODE содержит 96 382 символа, взятых их мировых шрифтов. Этих символов более чем достаточно для общения на всех известных языках мира, а также для написания классических (исторических) шрифтов многих языков. UNICODE включает в себя шрифты европейских алфавитов, средне-азиатское письмо, направленное справа налево, шрифты Азии, и многие другие. Подмножество символов (code points) HUN включает 70 207 идеографических символов определяемых по национальным и

промышленным стандартам Китая, Японии, Кореи, Тайвани, Вьетнама и Сингапура. Более того, UNICODE содержит знаки пунктуации, математические символы, технические символы, геометрические формы и графические метки (dingbats), фонетические знаки.

Ниже приведена сравнительная таблица кодов **ASCII** и **UNICODE**, взятая из Фрагмента спецификации UNICODE 4.0 (Unicode Standard, Version 4.0), размещенного на сайте Unicode Consortium.

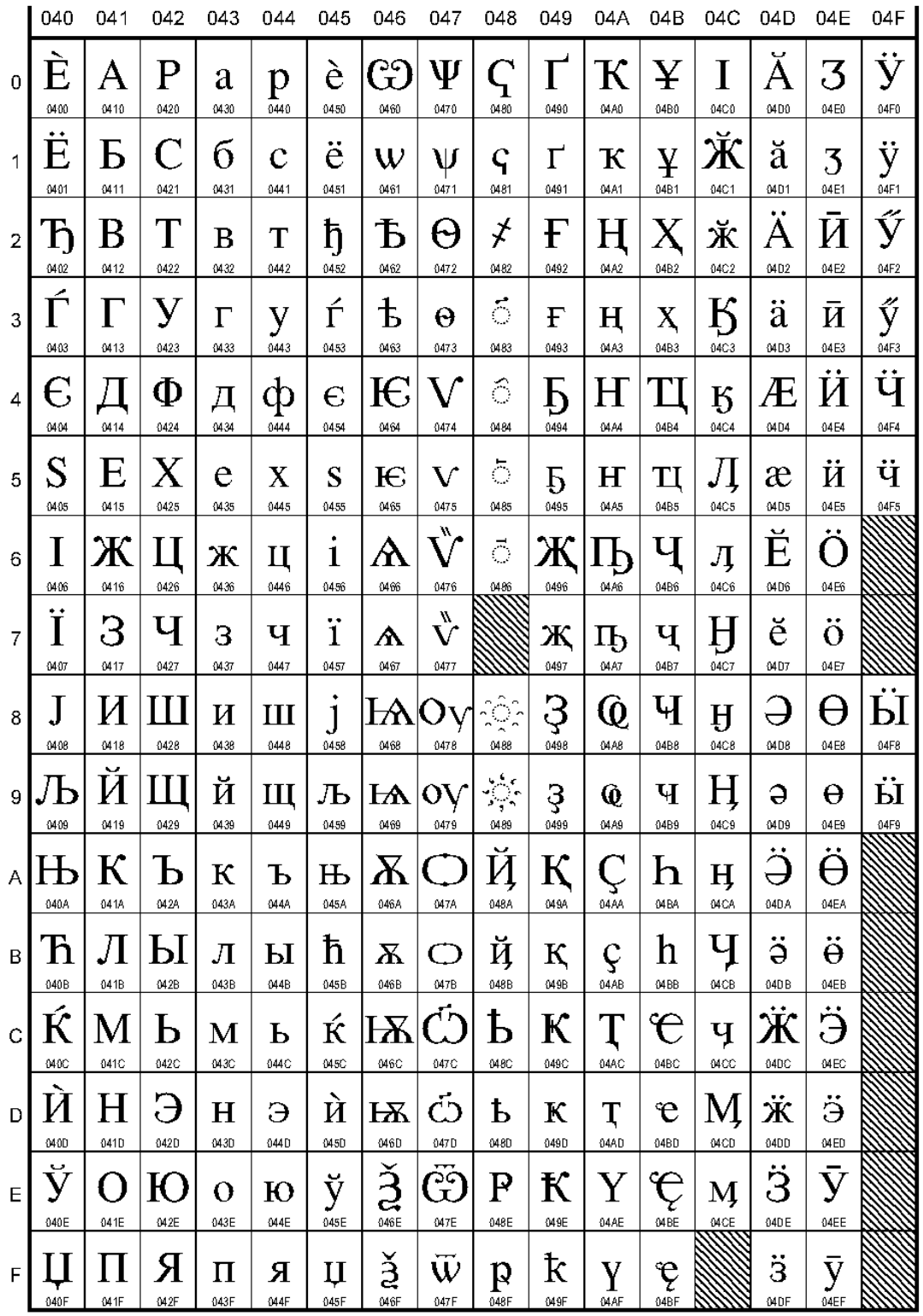
1.1 Coverage *Introduction*

Figure 1-1. Wide ASCII

ASCII/8859-1 Text		Unicode Text	
A	0100 0001	A	0000 0000 0100 0001
S	0101 0011	S	0000 0000 0101 0011
C	0100 0011	C	0000 0000 0100 0011
I	0100 1001	I	0000 0000 0100 1001
I	0100 1001	I	0000 0000 0100 1001
/	0010 1111		0000 0000 0010 0000
8	0011 1000	天	0101 1001 0010 1001
8	0011 1000	地	0101 0111 0011 0000
5	0011 0101		0000 0000 0010 0000
9	0011 1001	س	0000 0110 0011 0011
-	0010 1101	ل	0000 0110 0100 0100
1	0011 0001	ا	0000 0110 0010 0111
	0010 0000	م	0000 0110 0100 0101
t	0111 0100		0000 0000 0010 0000
e	0110 0101	α	0000 0011 1011 0001
x	0111 1000	≤	0010 0010 0111 0000
t	0111 0100	γ	0000 0011 1011 0011

Кодовая таблица для кириллицы приведена на следующем рисунке (взято из Фрагмента спецификации UNICODE 4.0 (Unicode Standard, Version 4.0), размещенного на сайте Unicode Consortium).

0400 **Cyrillic** **04FF**



The Unicode Standard 4.0, Copyright © 1991–2003, Unicode, Inc. All rights reserved.

Практические занятия

Тема 7: Представление информации в ПЭВМ типа IBM PC/AT

Варианты заданий

1. Представить следующие числа как двоичные с фиксированной точкой. Для записи выделяется слово.

а) $-1547_{(10)}$; б) $10001100011_{(2)}$; в) $-D17_{(16)}$;

г) $-1A4_{(16)}$; д) $-148_{(10)}$; е) $-100000011_{(2)}$;

ж) $347_{(8)}$; з) $-5554_{(8)}$; и) $123_{(16)}$.

2. Представить следующие числа как двоичные с фиксированной точкой без знака. Для записи выделяется слово.

а) $34547_{(10)}$; б) $100011111100011_{(2)}$;

в) $41A4_{(16)}$; г) $11148_{(10)}$;

д) $63471_{(8)}$; е) $5541_{(8)}$.

3. Представить следующие числа как двоичные с плавающей точкой в формате double.

а) $-17{,}56 \cdot 10^{4}{}_{(10)}$; б) $11{,}1000101 \cdot 10^{-11}{}_{(2)}$;

в) $0{,}234 \cdot 10^{-3}{}_{(10)}$; г) $0{,}100011101 \cdot 10^{100111}{}_{(2)}$;

д) $-13{,}17 \cdot 10^{4}{}_{(10)}$; е) $-A{,}71 \cdot 10^{3}{}_{(16)}$;

ж) $-10001{,}100011 \cdot 10^{10111}{}_{(2)}$; з) $0{,}0001 \cdot 10^{-12}{}_{(16)}$.

4. Представить следующие числа как двоичные с плавающей точкой в формате float.

а) $-17{,}56 \cdot 10^{4}{}_{(10)}$; б) $11{,}1000101 \cdot 10^{-11}{}_{(2)}$;

в) $0{,}234 \cdot 10^{-3}{}_{(10)}$; г) $0{,}100011101 \cdot 10^{100111}{}_{(2)}$;

д) $-13{,}17 \cdot 10^{4}{}_{(10)}$; е) $-A{,}71 \cdot 10^{3}{}_{(16)}$;

ж) $-0{,}0001 \cdot 10^{-12}{}_{(16)}$; з);$10001{,}100011 \cdot 10^{10111}{}_{(2)}$

5. Интерпретировать содержимое четырех байт памяти как:
   - целое число со знаком;
   - целое число без знака;
   - вещественное число с плавающей точкой.

а)

1	**01100110**	**11100011111100110011010**
31	30 23	22 0

б)

1	**11100100**	**00111011000000110011011**
31	30 23	22 0

в)

0	**00000011**	**10101111001100110011111**
31	30 23	22 0

г)

0	**10001010**	**00000011000000110011000**
31	30 23	22 0

д)

0	**11000110**	**10110000111100110111111**
31	30 23	22 0

е)

0	**00000100**	**00000011000000110011000**
31	30 23	22 0

ж)

1	**10010111**	**10111111111100110111111**
31	30 23	22 0

з)

0	**11100010**	**00000011011110110011000**
31	30 23	22 0

и)

1	**01000010**	**00000011100000110111111**
31	30 23	22 0

к)

0	**00000000**	**10000011100000110110000**
31	30 23	22 0

л)

1	**10000000**	**10000000000000110000000**
31	30 23	22 0

Основная учебная литература:

1. Варфоломеева, Т.Н. Структуры данных и основные алгоритмы их обработки [Электронный ресурс] : учебное пособие / Т.Н. Варфоломеева. - Электрон. дан. - Москва : ФЛИНТА, 2017. - 159 с. - Режим доступа: https://e.lanbook.com/book/104903

2. Сундукова, Т.О. Структуры и алгоритмы компьютерной обработки данных [Электронный ресурс] : учебное пособие / Т.О. Сундукова, Г.В. Ваныкина. - Электрон. дан. - Москва : , 2016. - 805 с. - Режим доступа: https://e.lanbook.com/book/100513.

3. Коврижных А.Ю. Основы алгоритмизации и программирования : практикум. В 2 ч. Ч. 1. Задачи и упражнения [Электронный ресурс] : учебно-методическое пособие / А.Ю. Коврижных, Е.А. Конончук,, Г.Е. Лузина. - Электрон. дан. - Екатеринбург : УрФУ, 2016. - 52 с. - Режим доступа: https://e.lanbook.com/book/98290 .

4. Коврижных, А.Ю. Основы алгоритмизации и программирования : практикум. В 2 ч. Ч. 2. Расчетные работы [Электронный ресурс] : учебно-методическое пособие / А.Ю. Коврижных, Е.А. Конончук, Г.Е. Лузина. - Электрон. дан. - Екатеринбург : УрФУ, 2016. - 44 с. - Режим доступа: https://e.lanbook.com/book/98291.

Дополнительная учебная литература:

1. Колдаев В.Д. Основы алгоритмизации и программирования [Текст] : учеб. пособие / В.Д. Колдаев ; ред. Л. Г. Гагарина. - М. : ФОРУМ-ИНФРА-М, 2015. - 413 с.

2. Семакин И.Г. Основы алгоритмизации и программирования [Текст] : учеб. для студентов СПО / И. Г. Семакин, А. П. Шестаков. - 2-е изд., стер. - М. : Academia, 2014. - 300 с.

3. Вирт Н. Алгоритмы и структуры данных [Текст] : научное издание / Н. Вирт ; пер. с англ. Д. Б. Подшивалов. - М. : Мир, 1989. - 360 с.

4. Кнут Д. Искусство программирования [Текст] / Д. Э. Кнут ; ред. Ю. В. Козаченко. - Испр. и доп. изд. - М.; СПб.; К. : Вильямс, 2000 - 2001. - (Классический труд). - Тит. л. рус., англ. - Т. 1 : Основные алгоритмы / пер.: С. Г. Тригуб, Ю. Г. Гордиенко, И. В. Красиков. - 3-е изд. - 2001. - 712 с.

5. Кнут Д. Искусство программирования [Текст] / Д. Э. Кнут ; ред. Ю. В. Козаченко. - Испр. и доп. изд. - М.; СПб.; К. : Вильямс, 2000 - 2001. - (Классический труд). - Тит. л. рус., англ. - Т. 2 : Получисленные алгоритмы / пер.: В. Т. Тертышный, Л. Ф. Козаченко, И. В. Красиков. - 3-е изд. - 2001. - 828 с.

6. Кнут Д. Искусство программирования [Текст] / Д. Э. Кнут ; ред. Ю. В. Козаченко. - Испр. и доп. изд. - М.; СПб.; К. : Вильямс, 2000 - 2001. - (Классический труд). - Тит. л. рус., англ. - Т. 3 : Сортировка и поиск / пер.: В. Т. Тертышный, И. В. Красиков. - 2-е изд. - 2000. - 822 с.

7. Ахо А. В. Структуры данных и алгоритмы [Текст] : пер. с англ. / А.В. Ахо, Д.Э. Хопкрофт, Д.Д. Ульман. - М. : Издательский дом "Вильямс", 2001. - 382 с.